香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

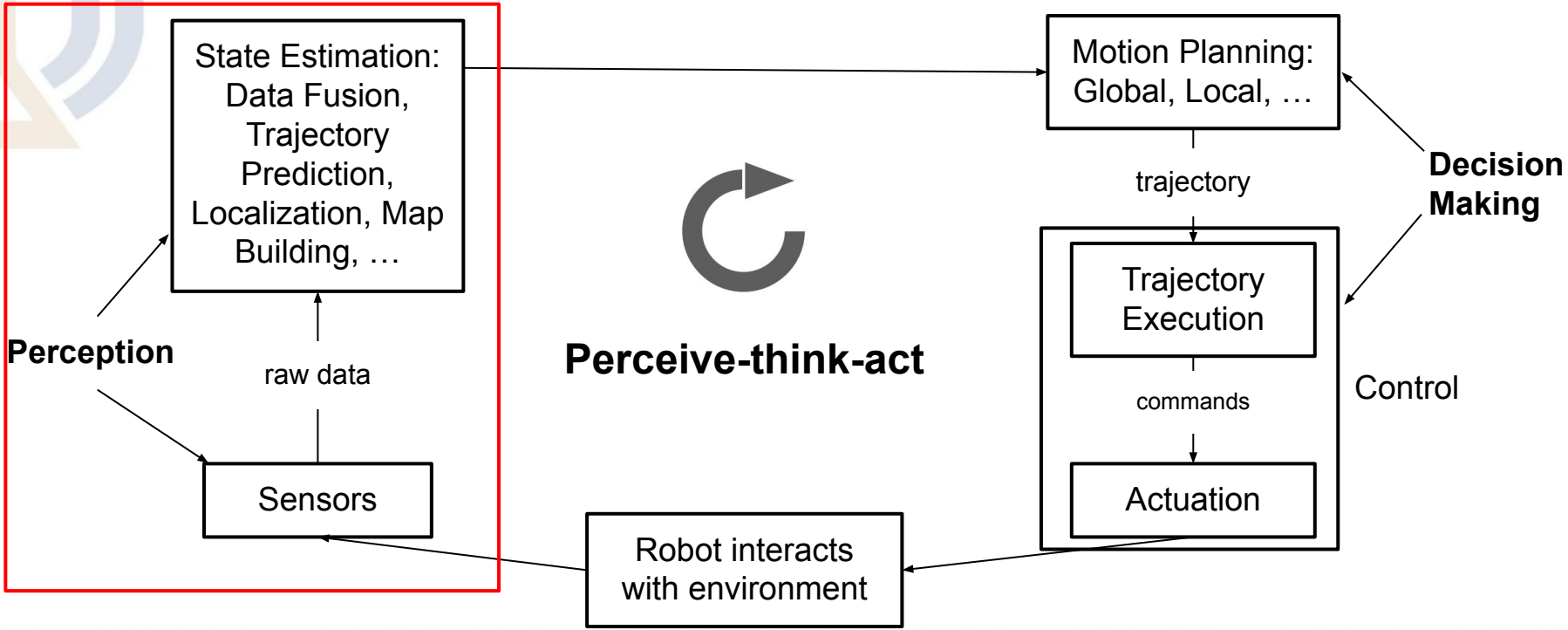
Embodied AI

Lecture 5-6: Object Detection and Tracking

Junwei Liang
AI Thrust
HKUST (Guangzhou)

Recap - The Autonomous AI Paradigm

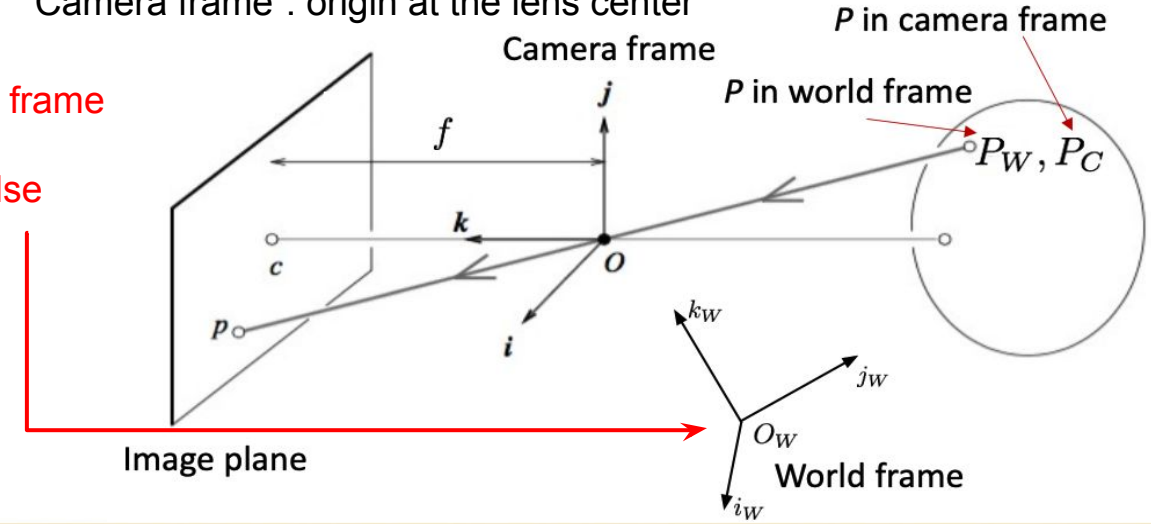
- The Perceive-think-act circle



Recap - Perspective Projection

- Goal: project world points onto the camera image
- Assumption: we have a pinhole camera model (all results also hold under thin lens model, assuming camera is focused at ∞)
  - “frame”: coordinate system
  - “Camera frame”: origin at the lens center

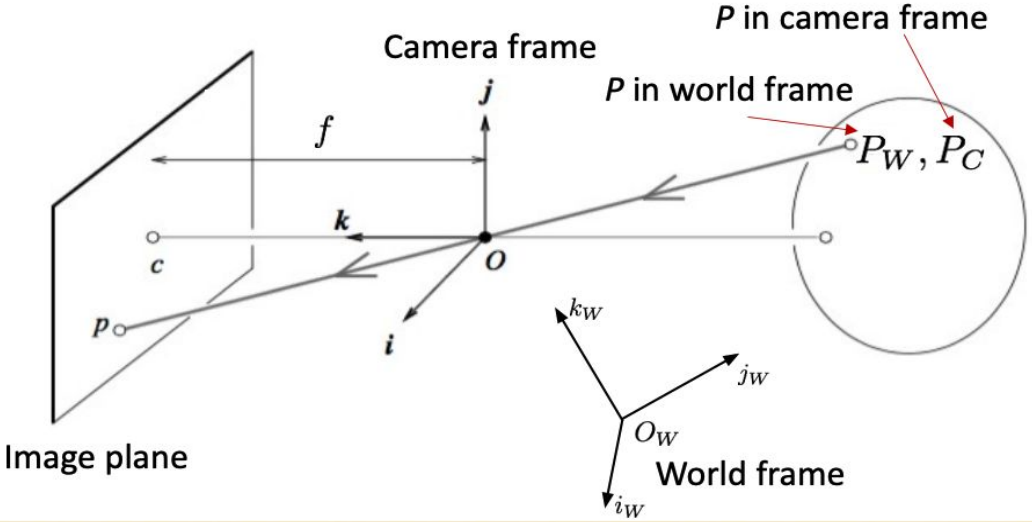
And the world frame has an origin somewhere else



Recap - Perspective Projection

- Steps
  - 1. Transform Pw into Pc (X, Y, Z)
  - 2. Map Pc into p (image plane, (x,y))
  - 3. Map p into (u, v) pixel coordinates

Pw and Pc are the same point, but in different frame (coordinate system) with different origins



Recap - Perspective Projection

- Projection can be equivalently written in homogeneous coordinates

Focal length + Principal point offset + the pixel–real-world scale

$$\underbrace{\begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

The XYZ in camera frame

The x,y in pixels

Camera matrix/
Matrix of intrinsic parameters

P_c in homogeneous coordinates

Homogeneous pixel coordinates

$$u = k_x \tilde{x} = \overbrace{k_x f}^{\alpha} \frac{X_C}{Z_C} + \overbrace{k_x \tilde{x}_0}^{u_0}$$

$$v = k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0}$$

$$p^h = [K \quad 0_{3\times 1}] P_C^h$$

Recap - Perspective Projection

- Projection can be equivalently written in homogeneous coordinates

Point p in homogeneous pixel coordinates

$$p^h = [K \quad 0_{3\times 1}]P_C^h$$

Point P_c in homogeneous camera coordinates

Recap - Perspective Projection

- Collecting all results

$$p^h = [K \quad 0_{3\times1}]P_C^h = K[I_{3\times3} \quad 0_{3\times1}]\begin{bmatrix} R & t \\ 0_{1\times3} & 1 \end{bmatrix} P_W^h$$

- Hence

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{K[R \quad t]}_{\text{Projection matrix } M} P_W^h \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Intrinsic parameters

Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Degree of freedom:
K: 4
Rt: 6

Recap - (Inverse) Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
  - Here is the full intrinsic in Carla

```
K = [[f, 0, Cu],
     [0, f, Cv],
     [0, 0, 1 ]]
```

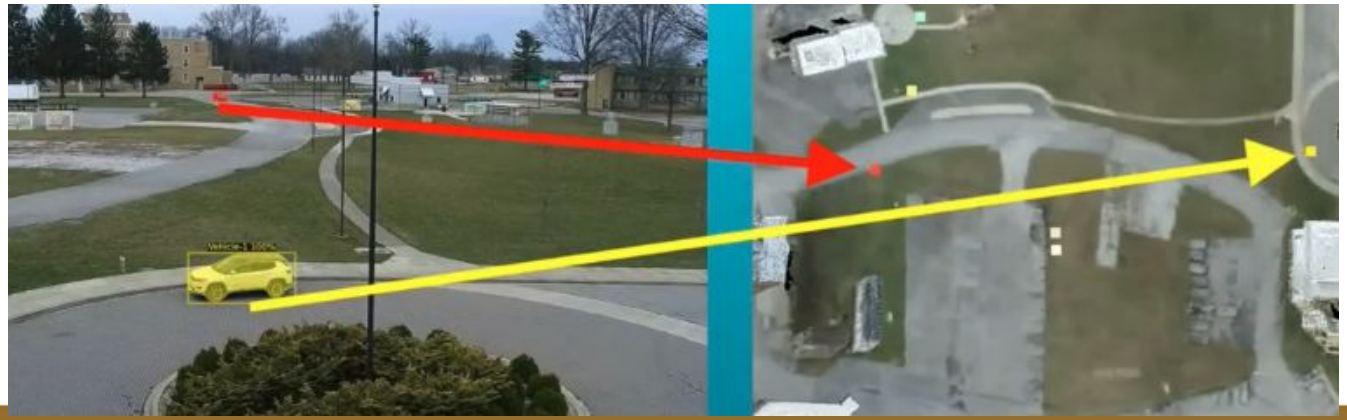
How to do inverse perspective projection?
Given p2d= [u, v, 1], how to get world point P = [X, Y, Z]?
Rt (camera extrinsic) is known in 3D simulation at all times

P = (inv(K)*p2d) * depth * Rt

https://github.com/JunweiLiang/Multiverse/blob/master/forking_paths_dataset/code/utils.py#L930

Recap - Bird-Eye-View Perception

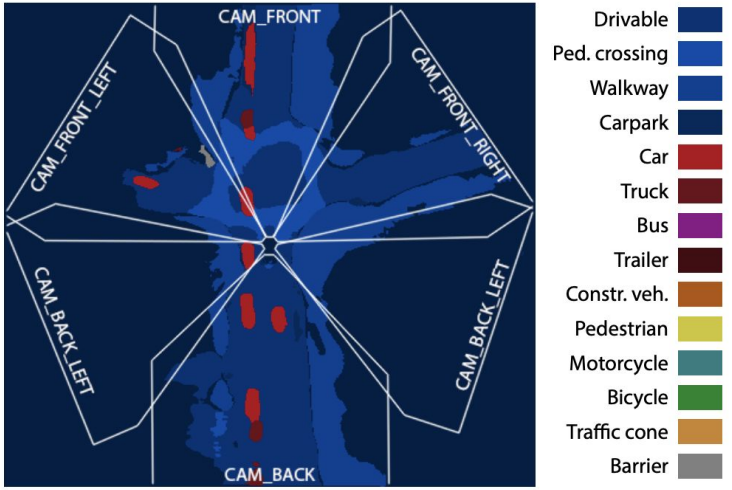
- If the camera is calibrated (intrinsic and extrinsic known), given image point p, can we get world point P in 3D world?
  - No! We can only know P is on the line OP
  - But we can still get from **image plane to the ground plane** (Bird Eye View!!)
    - Need to know the ground plane's parameters
    - Homography matrix = $K_2 * R_{21} * inv(K_1)$
    - This allow you to..



Bird-Eye-View Perception

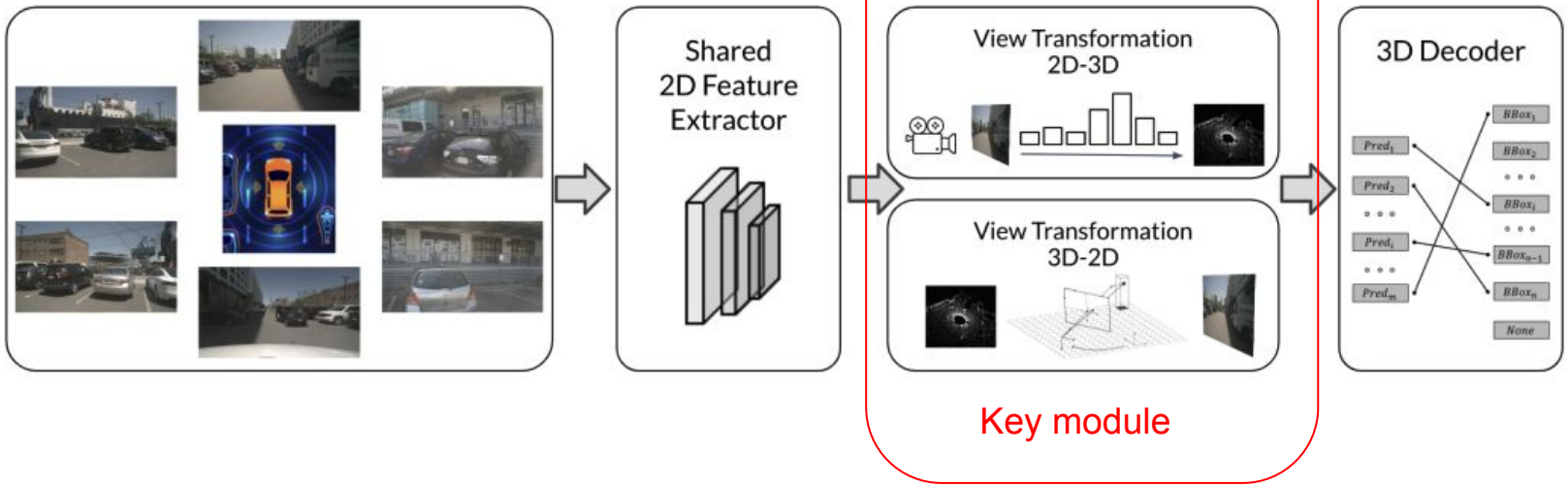
- What is BEV perception?
  - Previously, detection/segmentation/tracking, etc., done in a front view or perspective view
    - With more cameras and sensors, need a unified view
  - From multi-camera/sensor to a top-down unified view (BEV)
  - Increasingly popular for autonomous driving algorithms





Bird-Eye-View Perception

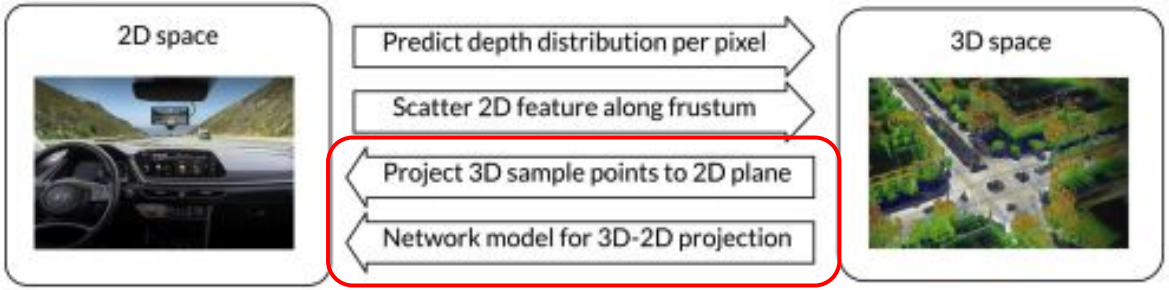
- General BEV pipeline for RGB cameras



[1] Delving into the Devils of Bird's-eye-view Perception: A Review, Evaluation and Recipe. ArXiv 2022

Bird-Eye-View Perception

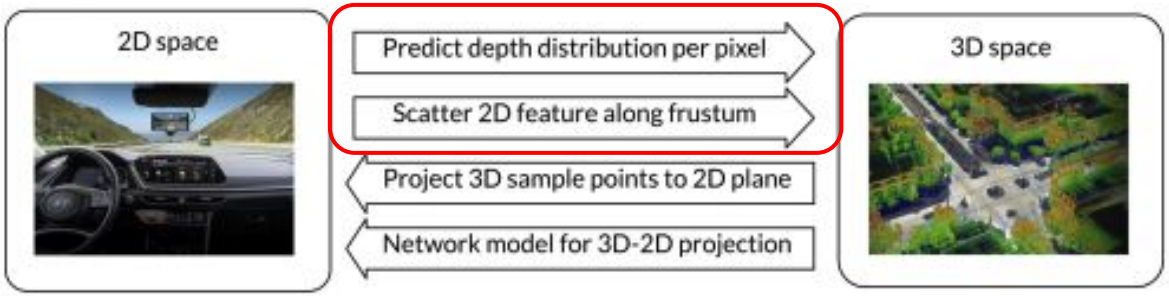
- General BEV pipeline for RGB cameras
  - 2D-3D or 3D-2D?



From 3D to 2D
1. On the BEV map, for each (3D) point, project to the camera 2D plane using camera parameters
2. Sample feature from 2D space

Bird-Eye-View Perception

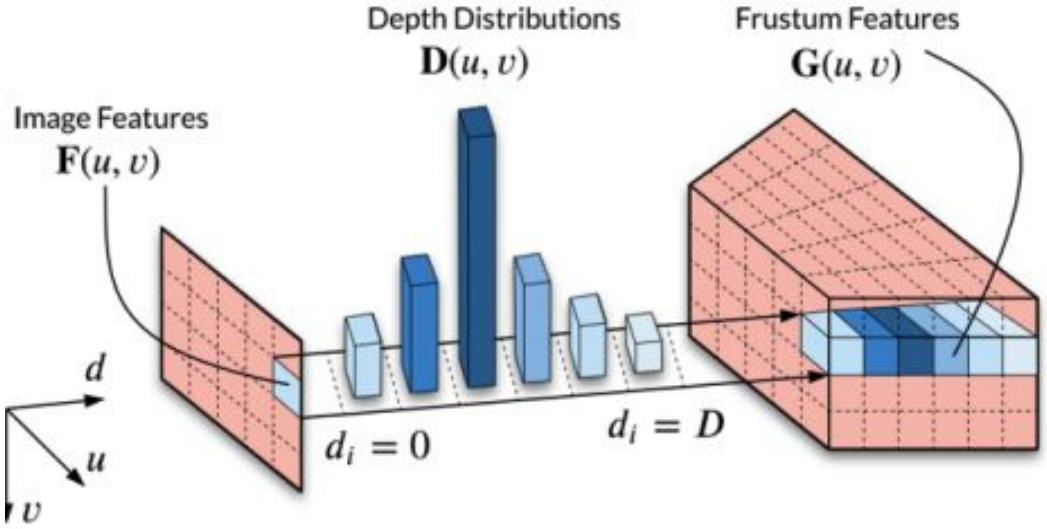
- General BEV pipeline for RGB cameras
  - 2D-3D or 3D-2D?



From 2D to 3D?
How do we get feature in 3D space directly?

Bird-Eye-View Perception

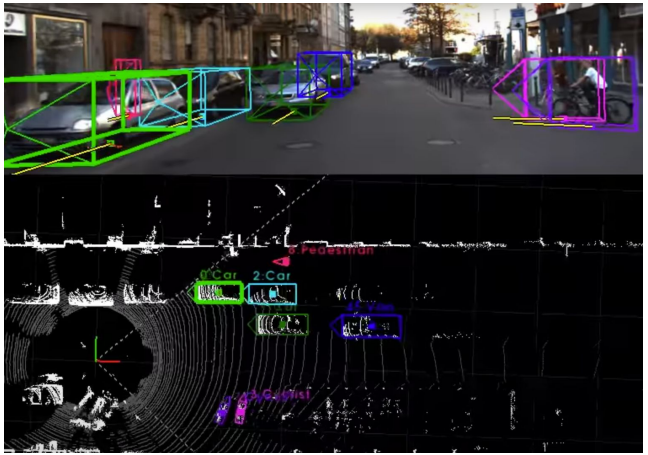
- General BEV pipeline for RGB cameras
  - 2D-3D
    - Predict per-pixel depth as feature distribution along the projection cone
    - 2D features -> 3D voxels

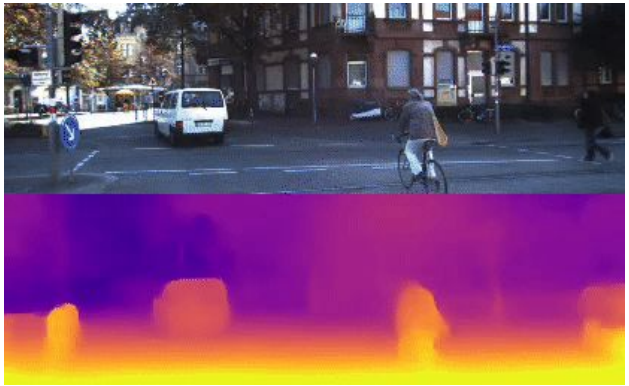


Bird-Eye-View Perception

- BEV Perception datasets
  - KITTI, - RGB frames, 3D bbox, Depth*



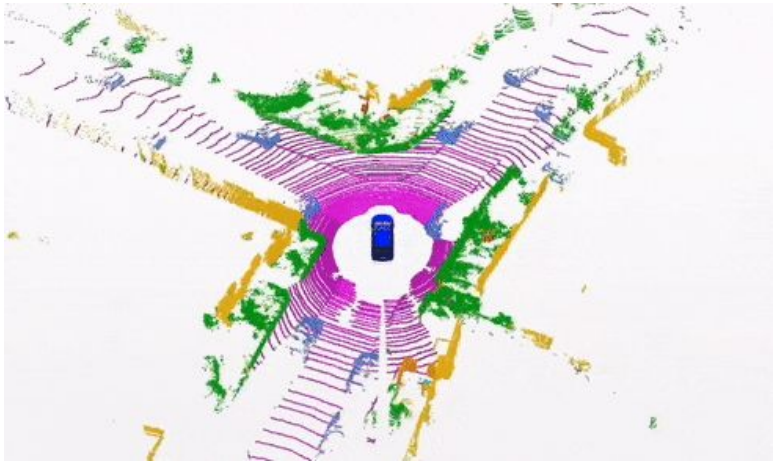




[1] Geiger, Andreas, Philip Lenz, and Raquel Urtasun. "Are we ready for autonomous driving? the kitti vision benchmark suite." CVPR 2012

Bird-Eye-View Perception

- BEV Perception datasets
  - SemanticKITTI - RGB frames, 3D bbox, 3D segmentation



[2] Behley, Jens, et al. "Semantickitti: A dataset for semantic scene understanding of lidar sequences." ICCV 2019

Bird-Eye-View Perception

- Existing BEV applications





Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - Object detection
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

Logistics: In-class Quizzes

- In-class Quizzes (25%)
  - Designed to complement the lectures and making sure you understand the key knowledge about the course
  - 6 individual assignment (so you need to complete this yourselves and no LLM allowed)
    - Grading based on your best 5 out of 6 (so you can miss one and still get full 100%)
  - **Will be posted on Canvas Quizzes section**
  - Schedule
    - 10:10 - 10:20 am.
    - Test 1: Sep 21 (Today)
    - Test 2: Oct 12
    - Test 3: Oct 26
    - Test 4: Nov 9
    - Test 5: Nov 23
    - Test 6: Nov 30

See the full schedule at course website:
https://hkust-aiaa4220.github.io/fall2026/lecs.html

Logistics: P1 Paper Presentation

- Project 1: Paper Presentation (25%)
  - You will select 1 paper from our list of Embodied AI research to present. This should show that you understand the paper, and you could present it to students who do not know the paper well. **You have 10 minutes to present and 5 minutes to handle at least one question (from TA/peers/teacher)**
    - Paper list: link
    - Full schedule
      - P1 presentation #1-8 from 10/12 to 11/30
        - (5-6 students per lecture) (starting at 10:10 am.)
  - This is an individual assignment
  - Project 1 is worth 25% credit
    - The grading is based on both the presentation (50%) and the slides (50%), by the teacher and TAs. Please follow the slide template here: link
    - You are required to submit the presentation slides after the presentation

Logistics: P1 Paper Presentation is Released

- Starting from Oct 12, 10:10-11:50
  - You have 10 minutes to present and 5 minutes to handle at least one question
  - Presentation slide template: link to pptx
  - Presentation order: Link
  - P1 grading rubrics: Link
  - Presentation slide submission (PDF): https://hkust-gz.instructure.com/courses/3918/assignments/28736
  - Announcement (about paper selection rule, time change rule): link
  - Oct 12:

AIAA4220 Paper Presentation — Final Schedule

This sheet updates automatically from Student Assignments. Paper choices may change; presentation times remain fixed unless students swap.

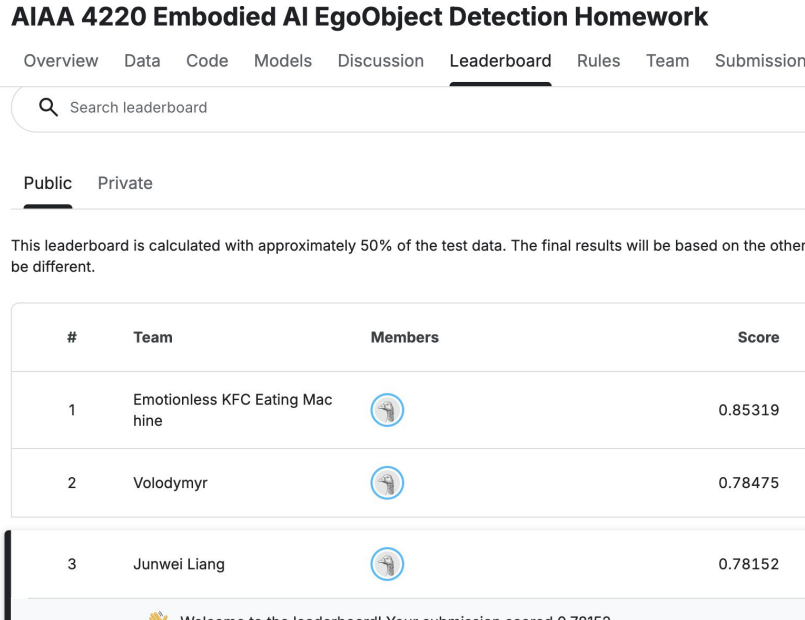
Presentation Date	Time	Name	er #	Theme	Paper Title
2026.10.12	10:20–10:35	Yuming ZHANG	23	Navigation & Social Navigation	ViNT: A Foundation Model for Visual Navigation
2026.10.12	10:35–10:50	Yilin WU	15	Robotic Manipulation & Affordance	RVT: Robotic View Transformer for 3D Object Manipulation
2026.10.12	10:50–11:05	Weihao LIU	37	3D & Egocentric Perception	3EED: Ground Everything Everywhere in 3D
2026.10.12	11:05–11:20	Fanxi JING	3	Robot Foundation Models / VLA	Exploring the Limits of Vision-Language-Action Manipulations in Cross-task Generalization
2026.10.12	11:20–11:35	Jiaxin DONG	18	Navigation & Social Navigation	FLUX: Accelerating Cross-Embodiment Generative Navigation Policies via Rectified Flow and Static-to-Dynamic Learning
2026.10.12	11:35–11:50	Chenhui LIU	2	Robot Foundation Models / VLA	DiT4DiT: Jointly Modeling Video Dynamics and Actions for Generalizable Robot Control

Logistics: P2 EgoObject Detection Kaggle Task

- Project 2: Machine Perception Homework (25%)
  - You will do an Ego-view object detection homework
  - This is an individual assignment
  - You will submit to a Kaggle leaderboard
    - Kaggle page with initial code instruction and dataset: link
    - When submitting your results to Kaggle, include **your name and student ID** in the submission description to ensure proper grading
  - You will also need to submit a project report and source code
    - Report template: link
  - **Grading based on model absolute score and project report**
    - Detailed grading rubrics: link
    - Three absolute score thresholds
      - S: full credit (0.9 mAP) / A: 80% (0.87 mAP) / B: 70% (0.8 mAP)

Logistics: P2 EgoObject Detection Kaggle Task

- Project 2: Machine Perception Homework (25%) - Schedule
  - Submit on the Kaggle leaderboard by Oct 26: link
    - You will see your results on public leaderboard; grading based on private leaderboard
  - Submit to Canvas by Oct 30: link
    - Zip PDF report and code into one file
  - Grading rubrics: link



Logistics: HPC for P2

- Courtesy of AI Thrust, we have prepared ¥200 HPC account for you to use for P2
  - How to use of HPC: https://hpc.hkust-gz.edu.cn/
  - How to get the account and password:
    - Send an email to me or the TAs with the subject [AIAA 4220 HPC Application] and we will provide you with the account and password
    - The cost limit is ¥200
      - About 100 GPU hours of A800
      - The ACD GPU is H100

The instructor (junweiliang@hkust-gz.edu.cn)
TA Jinghao (jhuang401@connect.hkust-gz.edu.cn)
TA Yuxin (yhe469@connect.hkust-gz.edu.cn)

HPC 三期智算集群资源定价

资源定价生效时间：2025 年 9 月 1 日

- 计算资源排队定价

资源类型	低档	中档	高档
GPU（ACD）	2.40 元/卡/时	3.75 元/卡/时	6.00 元/卡/时

注：
1、开启了 MIG 模式的 GPU 资源，作业 GPU 费用将按申请的显存容量计算。
2、作业/容器运行时间按实际启动、结束时间计算；运行时间不足 1 小时的，按实际运行秒数计算。
3、"低档"对应 SLURM 低优先级队列和容器开发环境资源，"中档"对应 SLURM 中优先级队列资源，"高档"对应 SLURM 高优先级队列资源。

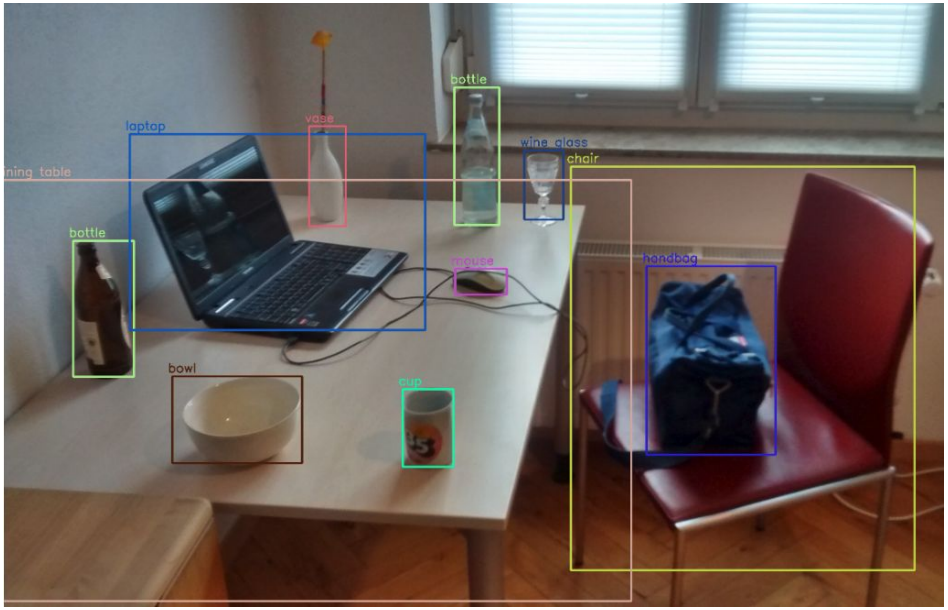


Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - **Object detection**
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

Object Detection

- Why object detection is important?
  - Core component in various computer vision tasks like facial recognition, image search, and augmented reality
  - Object detection is widely used in security systems for monitoring and tracking people or objects of interest
  - Object detection is also crucial for robots!

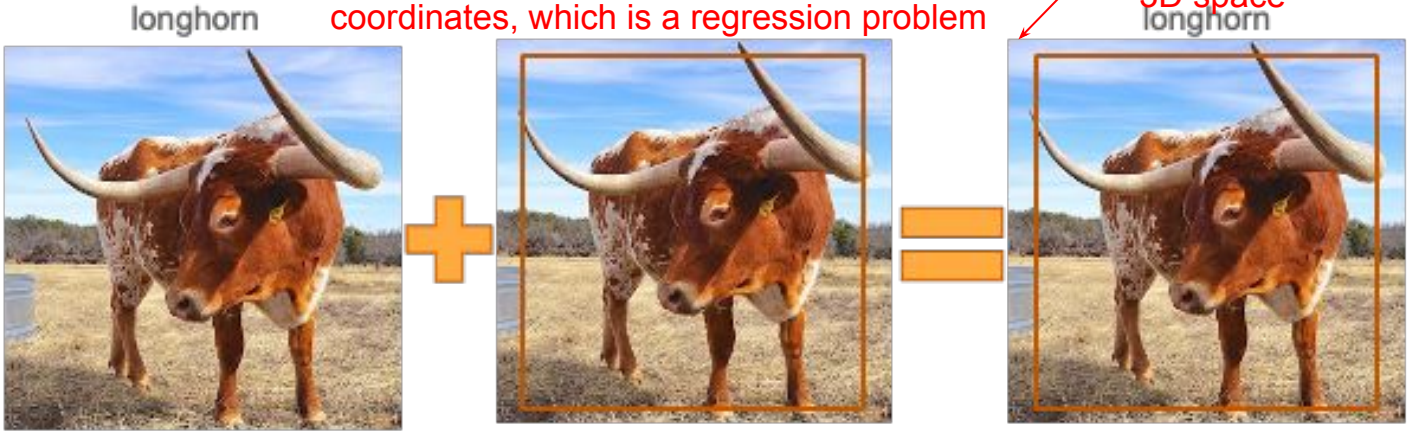


Object Detection - The Task

- Classification + Localization

In pixel space, xy coordinates of the top-left corner is (0, 0). Remember how we map this to 3D space

For localization, the model needs to output xy coordinates, which is a regression problem



We want to get boxes with classes

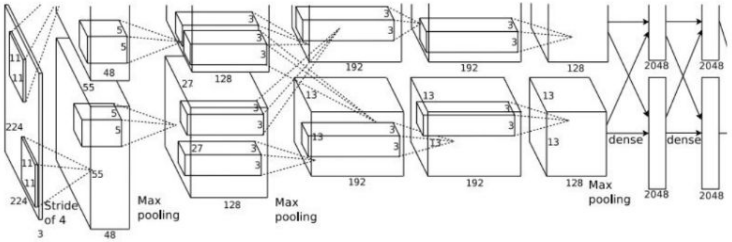
	classification	**localization**	**bbox detection**
output	class	(xyxy) or (xywh)	(xyxy, cls) or (xywh, cls)

Two ways to represent bounding boxes: xy of top-left and bottom-right/width-height of boxes

Object Detection - Multiple Objects

- What people come up with first - Sliding window approach
  - Apply a CNN to many different crops of the image, CNN classifies each crop as object or background





Dog? NO
Cat? NO
Background? YES

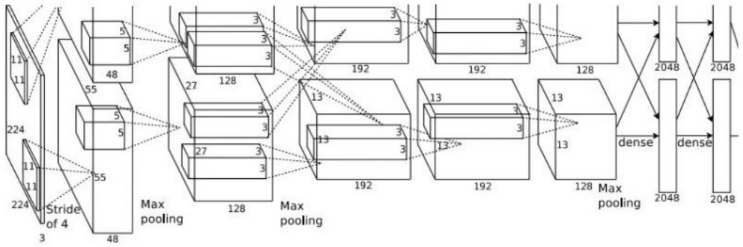
We can use a predefined bounding box to slide through the image.
Each time we crop the image and forward it to the CNN

* Credit: Jiajun Wu

Object Detection - Multiple Objects

- What people come up with first - Sliding window approach
  - Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



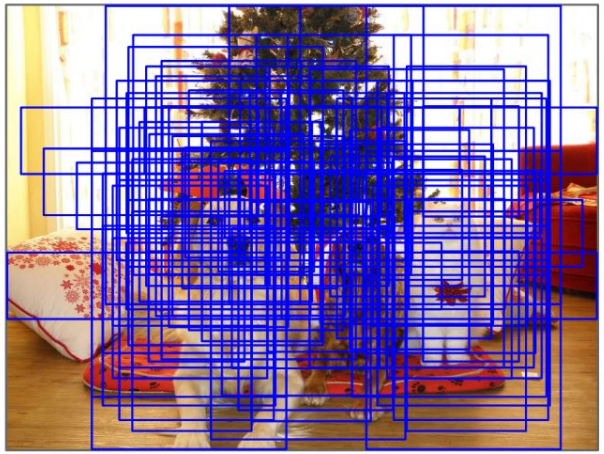


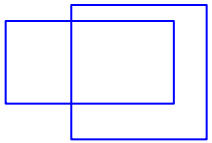
Dog? YES
Cat? NO
Background? NO

If the box happens to crop the object, the CNN should classify it correctly

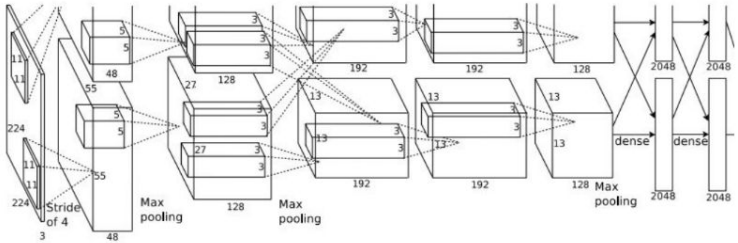
Object Detection - Multiple Objects

- What people come up with first - Sliding window approach
  - Apply a CNN to many different crops of the image, CNN classifies each crop as object or background
  - **Problem: very inefficient! Too many predefined boxes (locations, scales, sizes, aspect ratios) to try. Brute force does not work**





Aspect ratios: 1:2 or 1:1, etc.



Dog? NO
Cat? YES
Background? NO

Object Detection - Multiple Objects

- Using Selective Search to get region proposals
  - Find “blobby” image regions that are likely to contain objects
    - Based on low-level simple features like color, textures





For example, you can group all white pixel or close (with some threshold) together

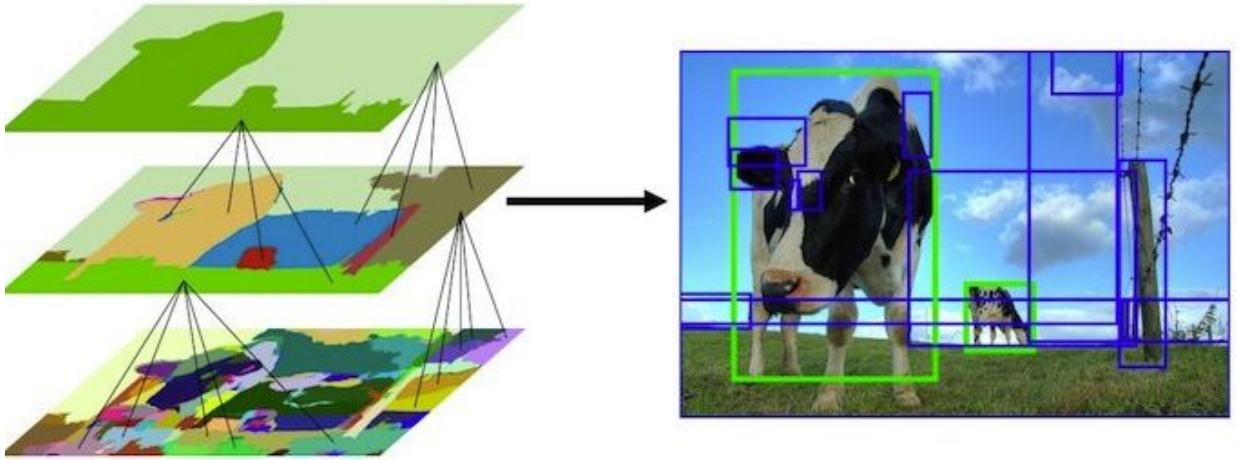
Over-segmented Image based on color blobs (group pixels with similar color)

Why over seg? Most of the actual objects contain 2 or more segmented parts. Start from the smallest and go up

*Measuring the objectness of image windows. TPAMI 2012

Object Detection - Multiple Objects

- Using Selective Search to get region proposals
  - Find “blobby” image regions that are likely to contain objects
    - Based on low-level simple features like color, textures
  - Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



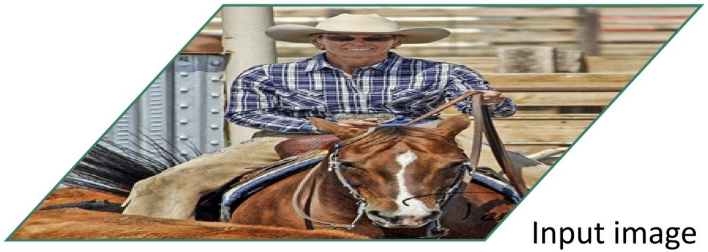
You can them draw bounding boxes based on the grouped pixels

*Segmentation as Selective Search for Object Recognition. ICCV 2011

Object Detection - Two-stage method

- R-CNN

Now we have a relatively fast way to get region proposals, we can now do object detection in a relatively practical setting.
Note: Imagine now you only have AlexNet CNN, etc.

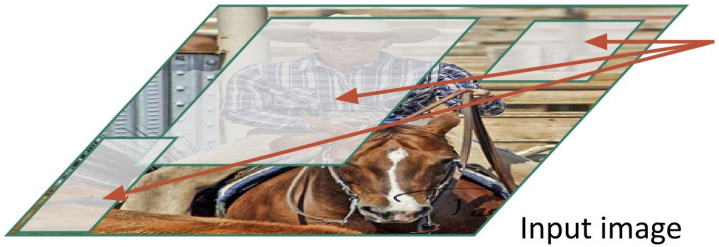


Input image

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN



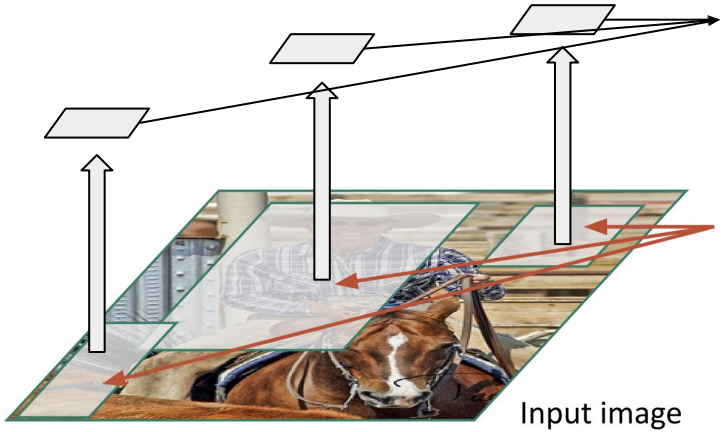
Input image

Regions of Interest (RoI) from a proposal method (~2k)

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN

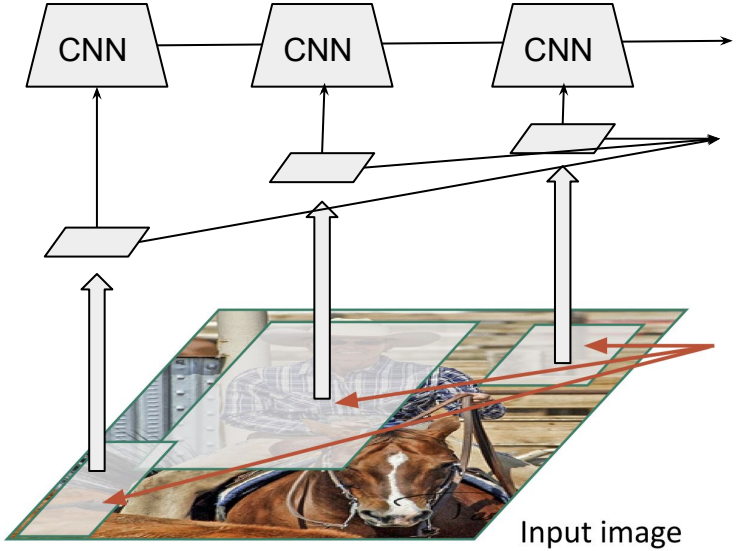


Why same size? We need same-sized features for the classification layer

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN



Each image is fed into a shared CNN and get a vector representation

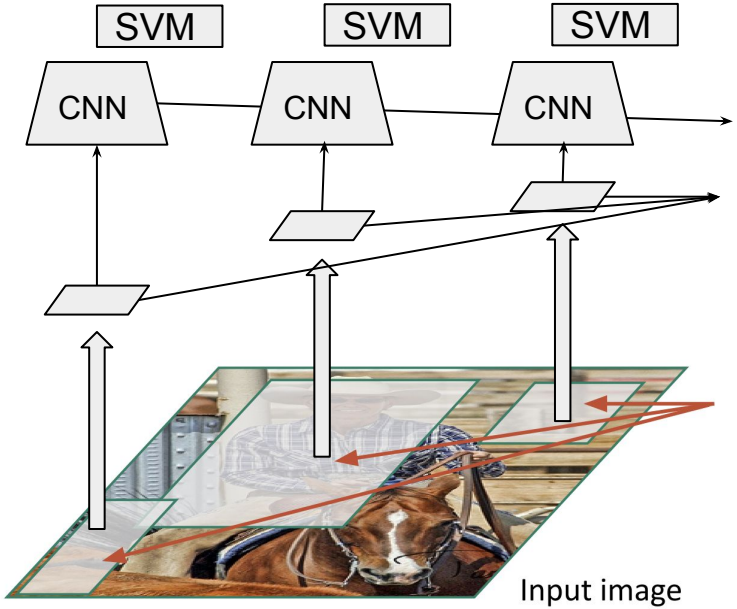
Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN



Use SVM to classify each region

Each image is fed into a shared CNN and get a vector representation

Warped image regions (224x224 pixels)

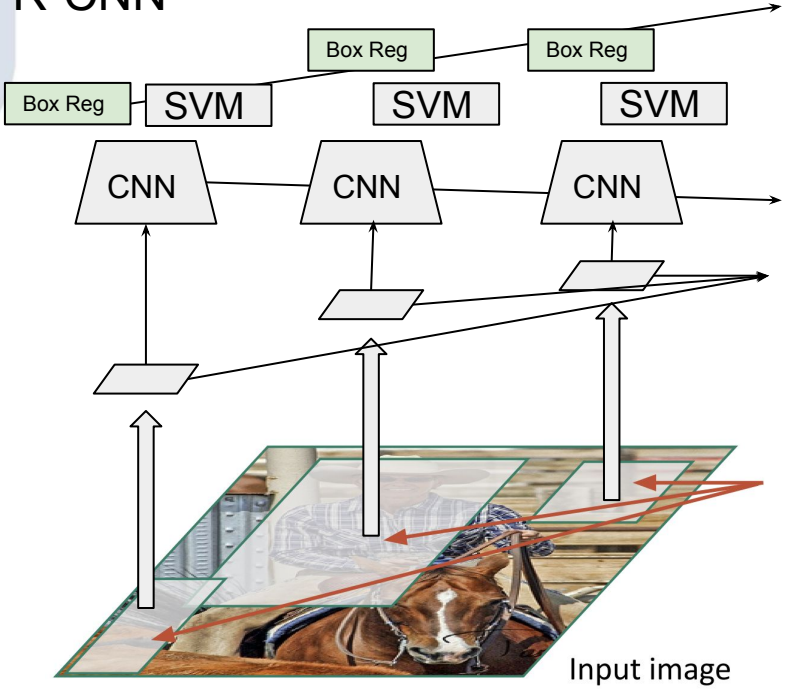
Regions of Interest (RoI) from a proposal method (~2k)

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN

This is key. We still need a box regression task to get the final box (the region proposals are not accurate)



Predict “corrections” to the RoI: 4 numbers: (dx, dy, dw, dh)

Use SVM to classify each region

Each image is fed into a shared CNN and get a vector representation

Warped image regions (224x224 pixels)

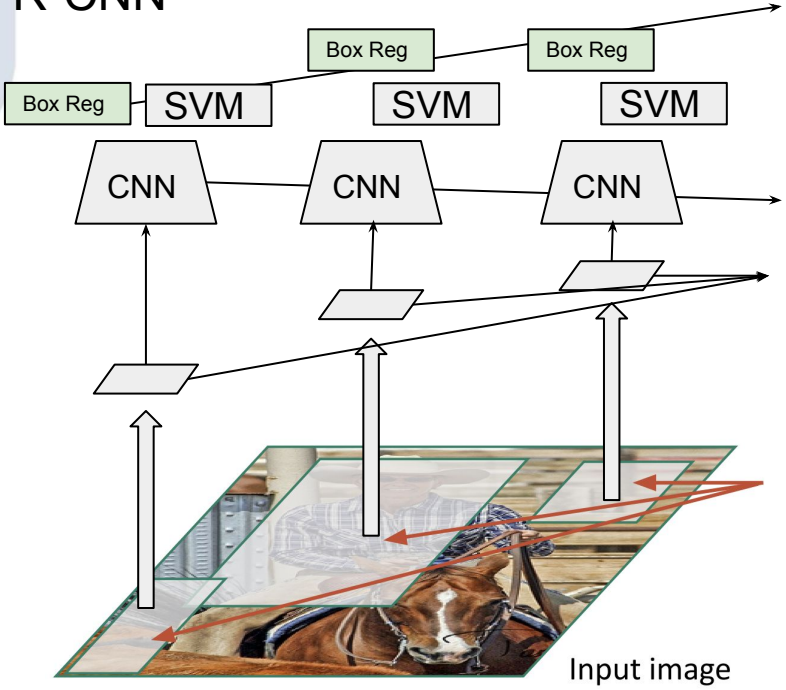
Regions of Interest (RoI) from a proposal method (~2k)

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN

This is key. We still need a box regression task to get the final box (the region proposals are not accurate)



Predict “corrections” to the RoI: 4 numbers: (dx, dy, dw, dh)

Use SVM to classify each region

Each image is fed into a shared CNN and get a vector representation

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

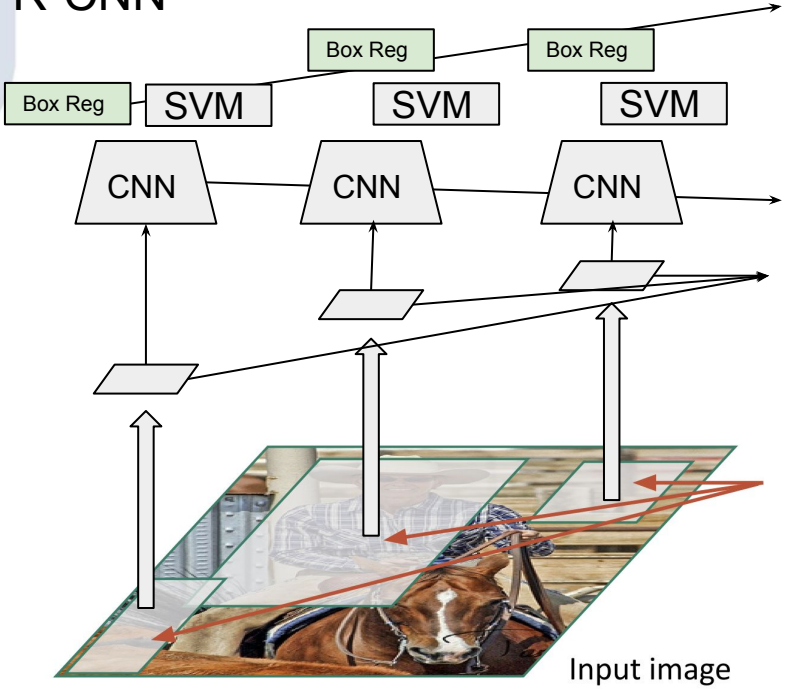
And that’s it! This is a 40k citation work. Given these tools, selective search, imagenet CNN, SVM, can you come up with it?

I’m sure you can. Just go with your intuition.

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN



Predict “corrections” to the RoI: 4 numbers: (dx, dy, dw, dh)

Use SVM to classify each region

Each image is fed into a shared CNN and get a vector representation

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

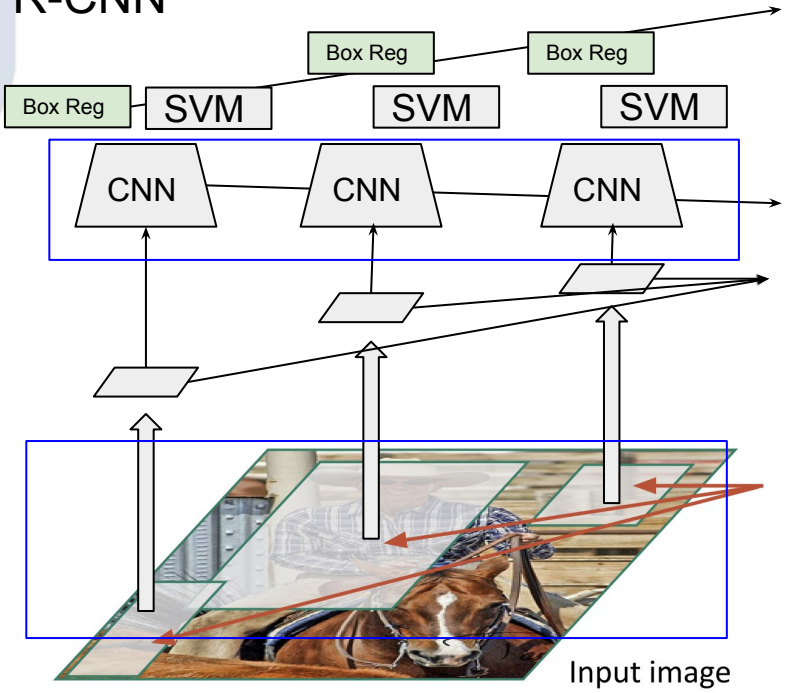
Problem: Very slow!
Where is the bottleneck?
Where do we waste compute?
How can we solve this?

Give me some 40k citation ideas. (if you have not read the fast R-CNN paper or the next slide)

*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- R-CNN



Predict “corrections” to the RoI: 4 numbers: (dx, dy, dw, dh)

Use SVM to classify each region

Each image is fed into a shared CNN and get a vector representation

Warped image regions (224x224 pixels)

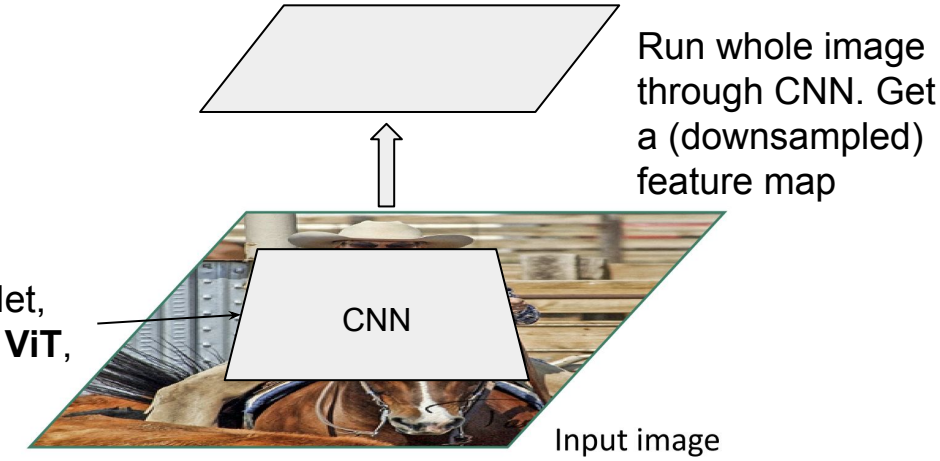
Regions of Interest (RoI) from a proposal method (~2k)

Idea: Regions are heavily overlapped and the CNN computation is redundant. So we compute the full-image CNN feature and crop the feature

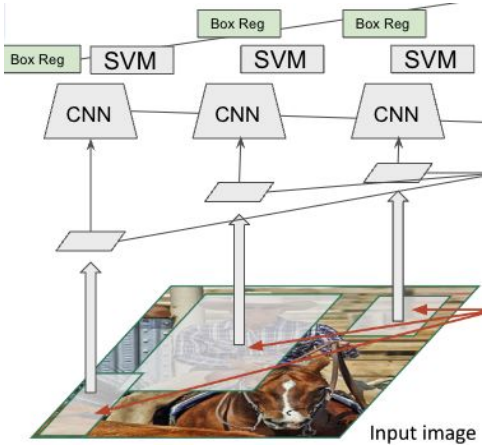
*Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR 2014

Object Detection - Two-stage method

- Fast R-CNN



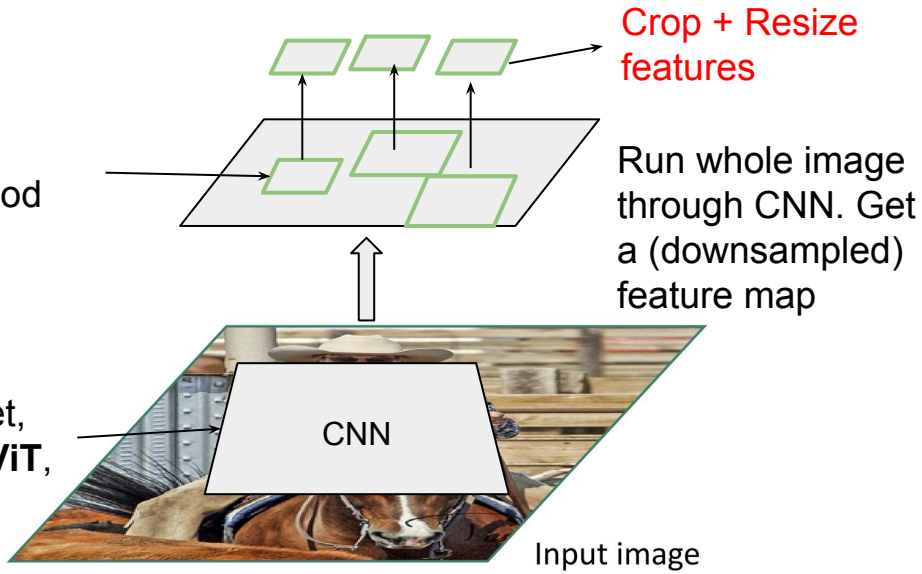
Slow R-CNN



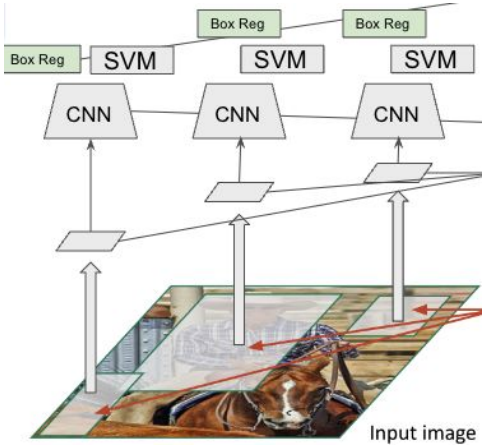
*Fast R-CNN. ICCV 2015

Object Detection - Two-stage method

- Fast R-CNN



Slow R-CNN

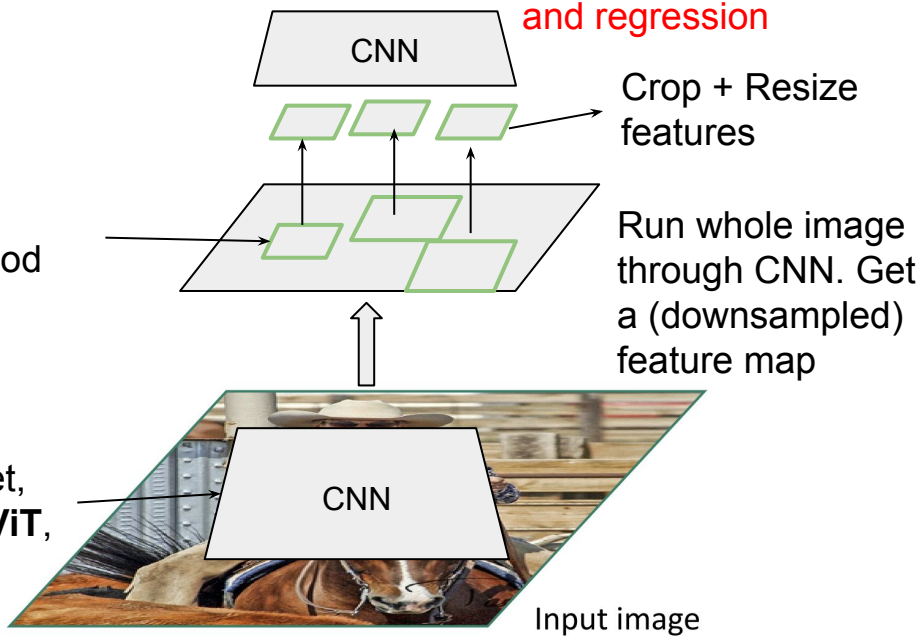


*Fast R-CNN. ICCV 2015

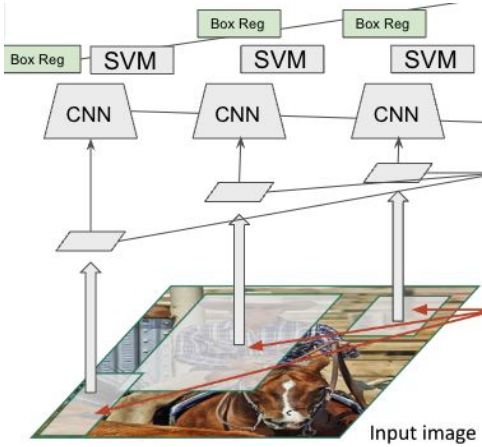
Object Detection - Two-stage method

- Fast R-CNN

Another CNN for box classification and regression



Slow R-CNN



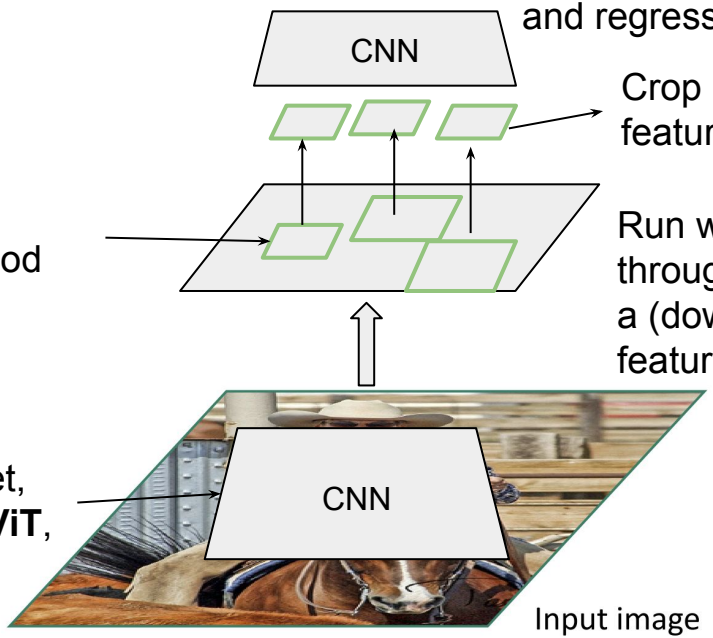
*Fast R-CNN. ICCV 2015

Object Detection - Two-stage method

- Fast R-CNN

Another CNN for box classification and regression

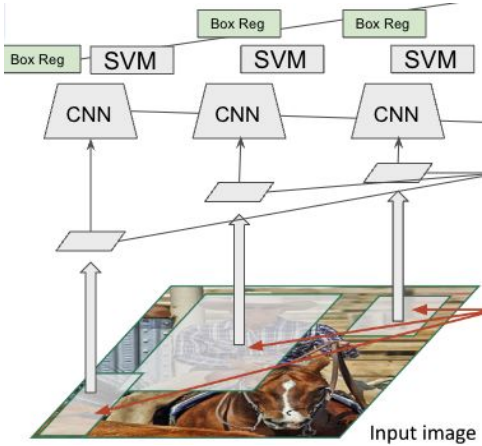
This CNN (usually 2-4 layers) is trained from scratch



"Backbone" network: AlexNet, VGG, ResNet, **ViT**, etc

This is pretrained from ImageNet

Slow R-CNN

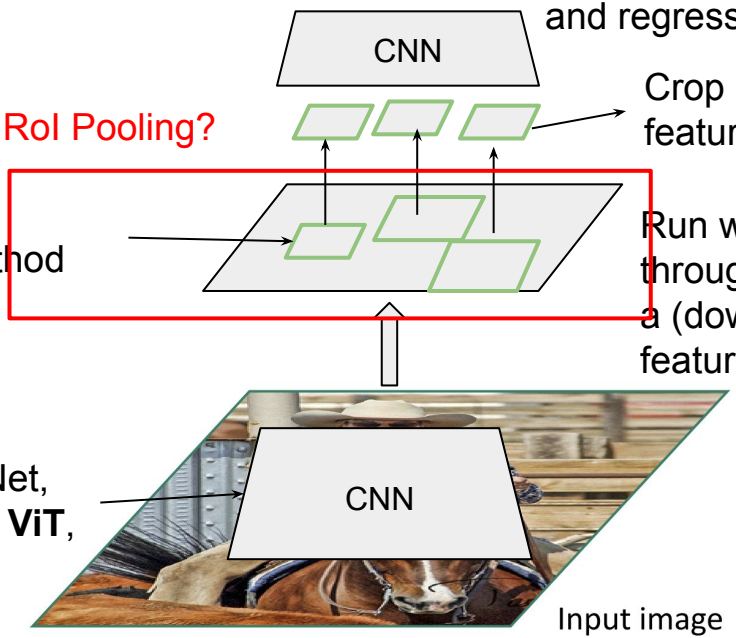


*Fast R-CNN. ICCV 2015

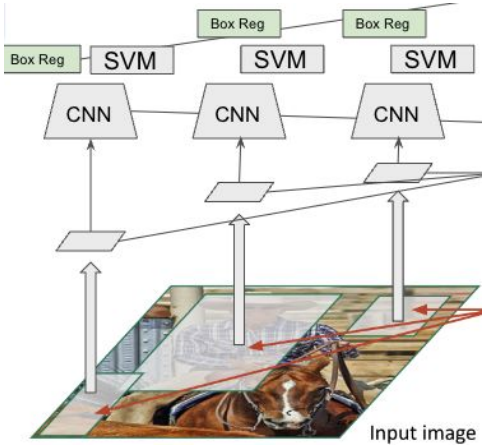
Object Detection - Two-stage method

- Fast R-CNN

Another CNN for box classification and regression



Slow R-CNN

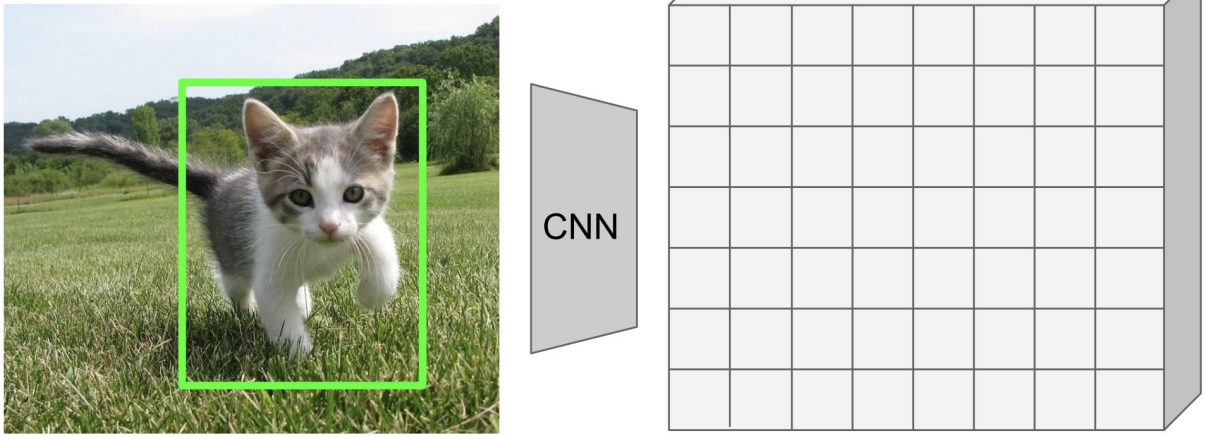


*Fast R-CNN. ICCV 2015

Object Detection - Two-stage method

- Cropping features: RoI Pool

Suppose the CNN feature downsampled the image from 640x480 to 20x15 (32x down is common)

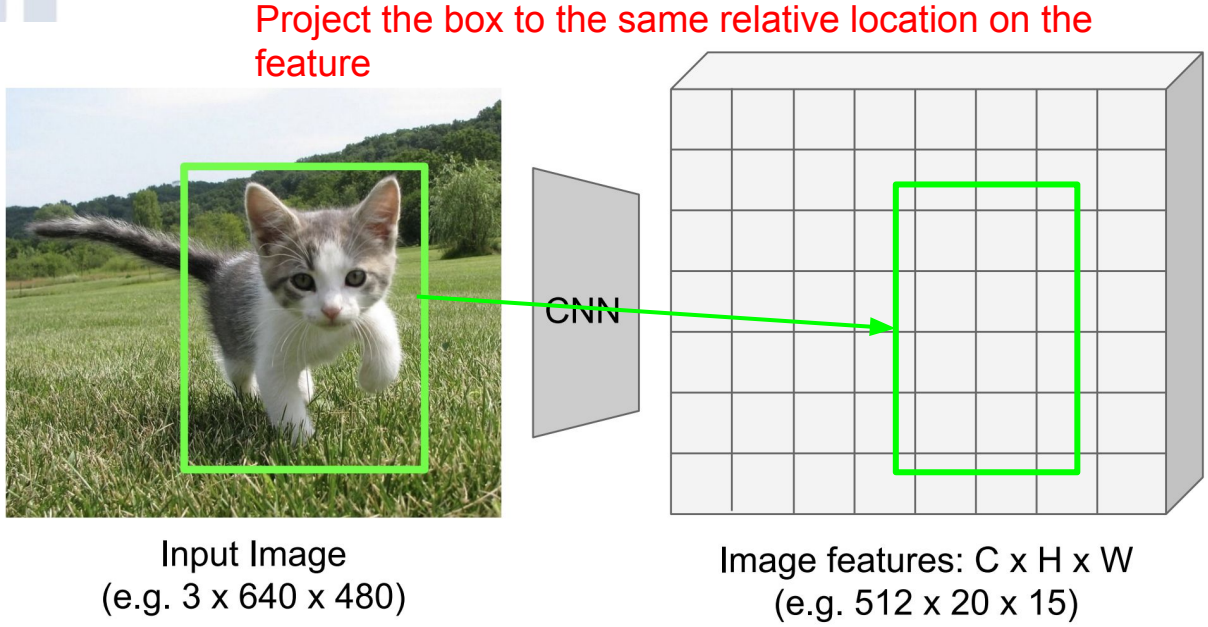


Input Image
(e.g. 3 x 640 x 480)

Image features: C x H x W
(e.g. 512 x 20 x 15)

Object Detection - Two-stage method

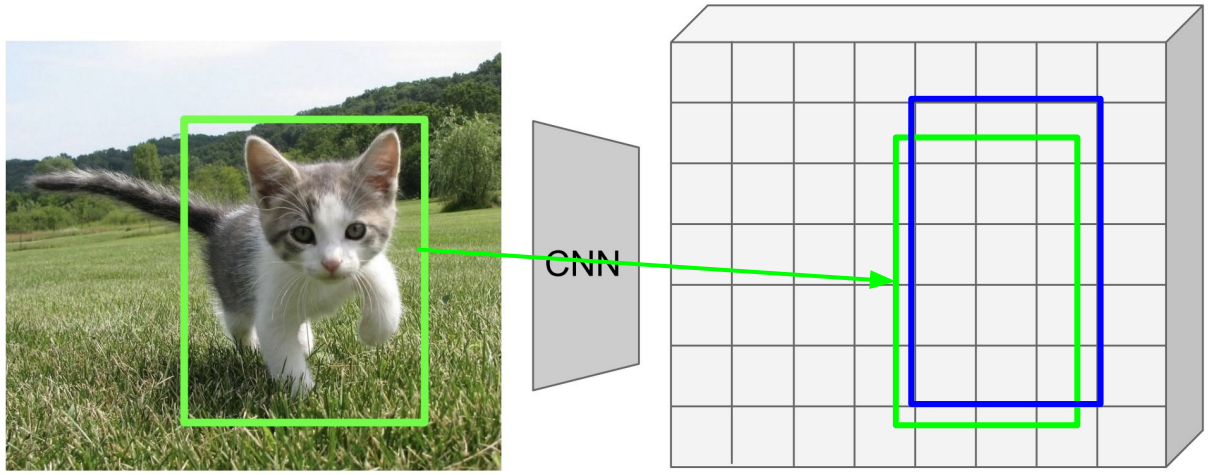
- Cropping features: RoI Pool



Object Detection - Two-stage method

- Cropping features: RoI Pool

"Snap" to the grid cells based on nearest neighbors. You can then simply use indexing (e.g. feature[a:b, c:d]) to take from the feature numpy array

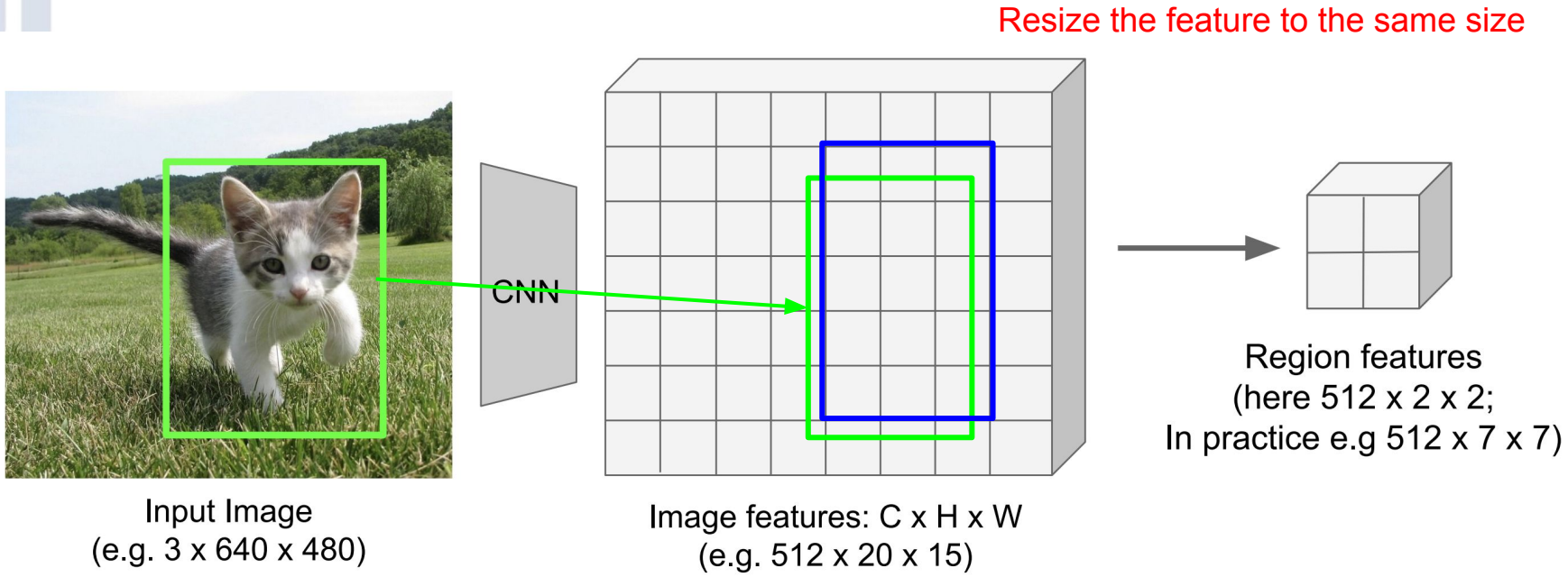


Input Image
(e.g. 3 x 640 x 480)

Image features: C x H x W
(e.g. 512 x 20 x 15)

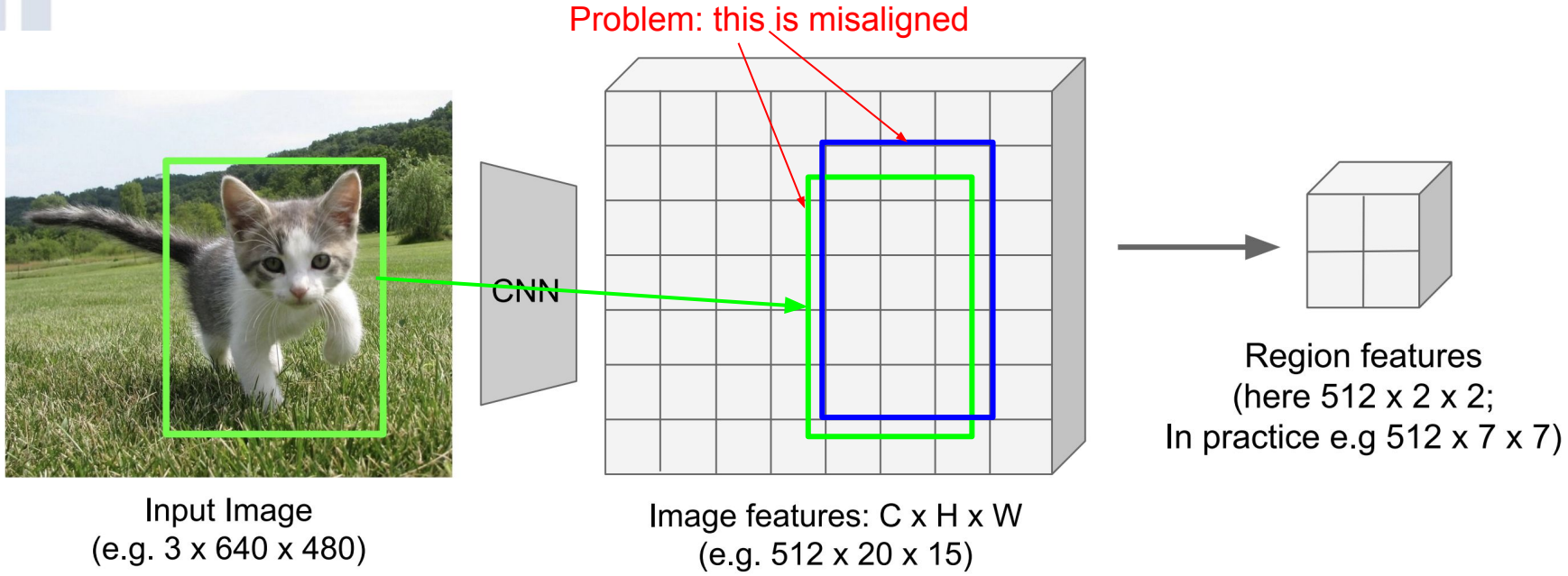
Object Detection - Two-stage method

- Cropping features: RoI Pool



Object Detection - Two-stage method

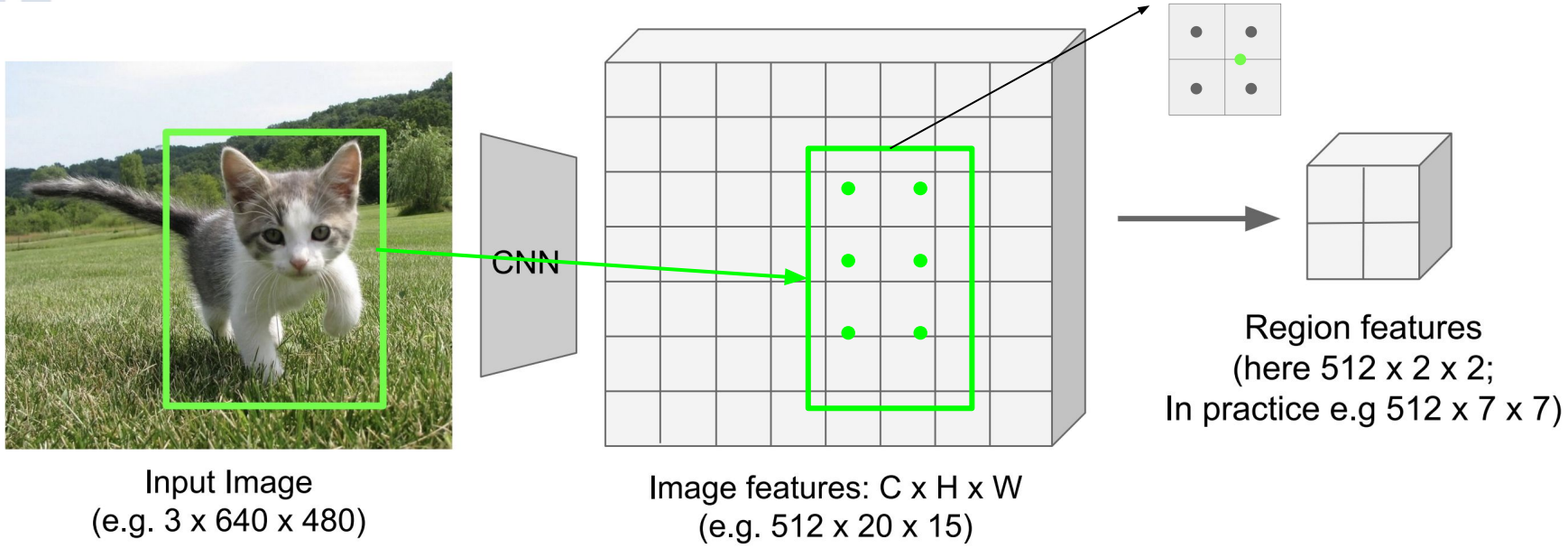
- Cropping features: RoI Pool



Object Detection - Two-stage method

- Cropping features: **RoI Align***

Perform bilinear interpolation, i.e. compute the green dot feature by weighted average of the surrounding four points based on L2 distance



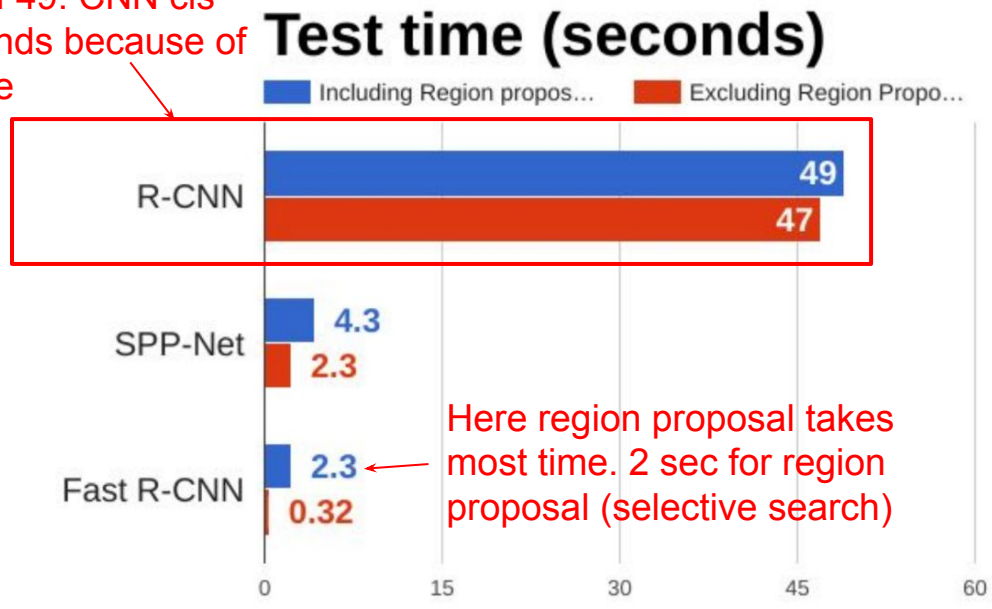
* Mask R-CNN. ICCV 2017

Object Detection - Two-stage method

- R-CNN vs. Fast R-CNN

Red bar: excluding region proposal time

Here region proposals takes 2 seconds out of 49. CNN cls takes 47 seconds because of waste compute



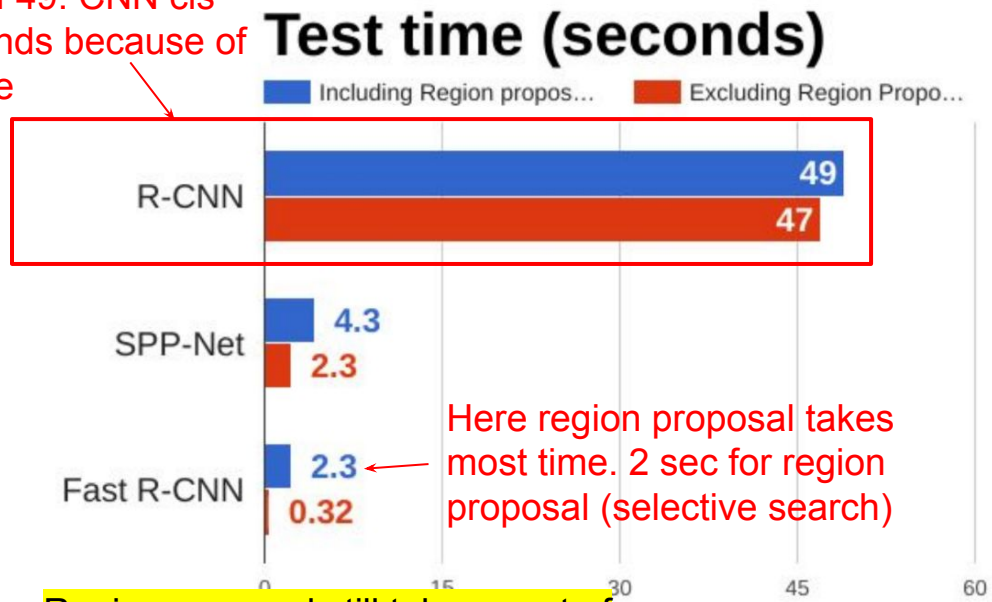
Here region proposal takes most time. 2 sec for region proposal (selective search)

Object Detection - Two-stage method

- R-CNN vs. Fast R-CNN

Red bar: excluding region proposal time

Here region proposals takes 2 seconds out of 49. CNN cls takes 47 seconds because of waste compute



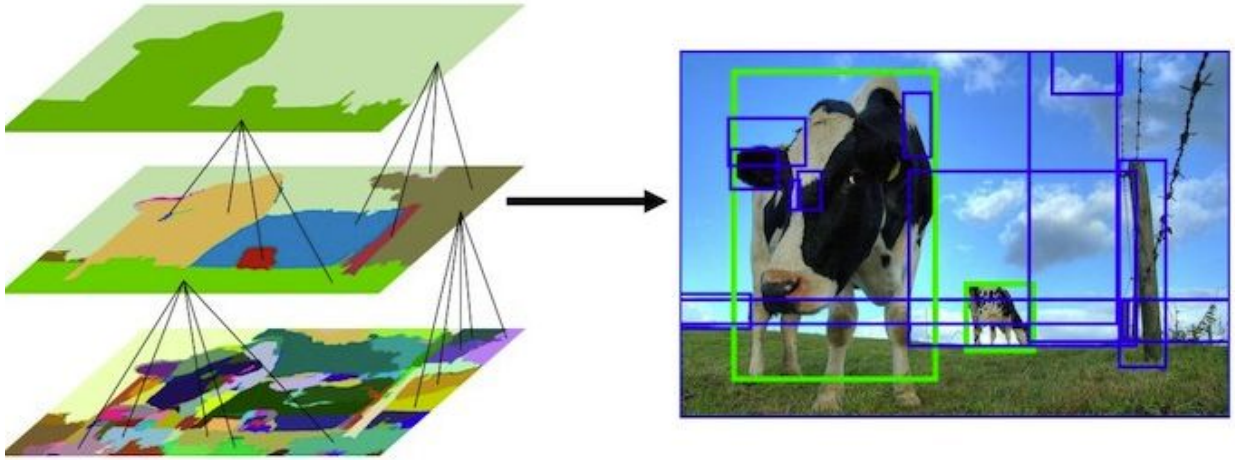
Here region proposal takes most time. 2 sec for region proposal (selective search)

Region proposal still takes most of the time. Let's improve this

Object Detection - Multiple Objects

- Using Selective Search to get region proposals
  - Selective search will need to **operate on pixels** to get segmentations first

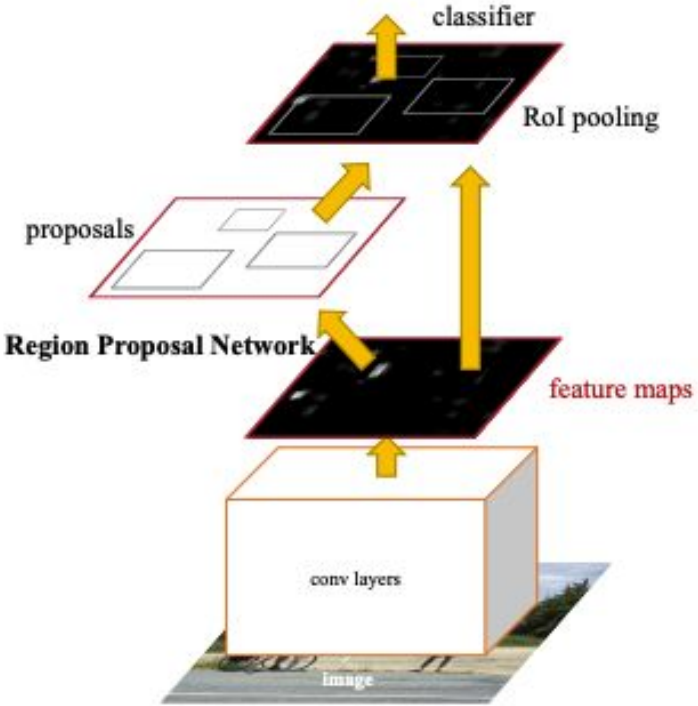
Why not use a CNN to do that? -> a 50k citation idea



*Segmentation as Selective Search for Object Recognition. ICCV 2011

Object Detection - Two-stage method

- Faster R-CNN
  - Use CNN features for region proposal!
  - Called Region Proposal Network (RPN)
  - Other parts the same as Fast R-CNN

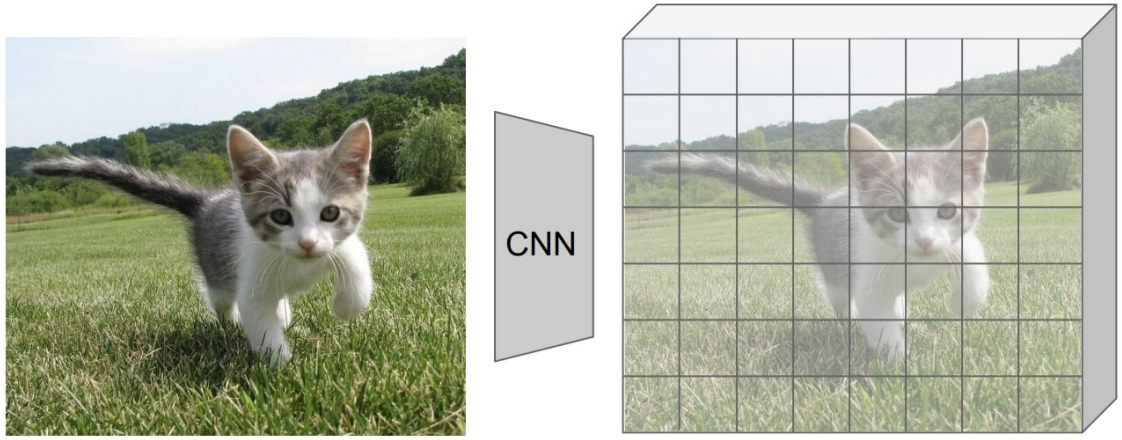


*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

- Region Proposal Network
  -

Suppose the CNN feature downsampled the image from 640x480 to 20x15



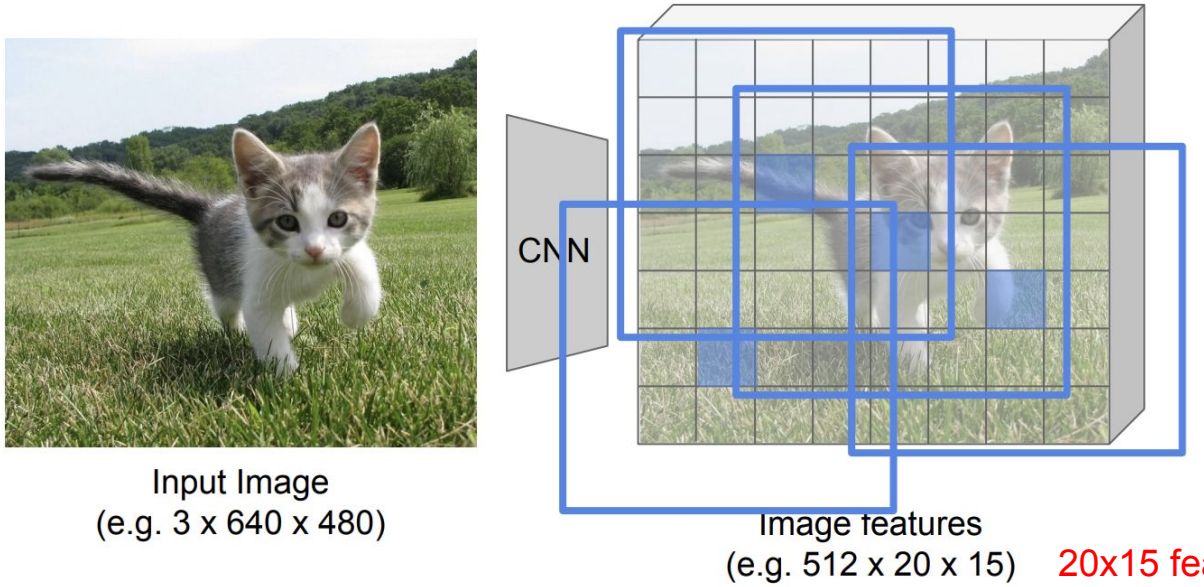
Input Image
(e.g. 3 x 640 x 480)

Image features
(e.g. 512 x 20 x 15)

*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

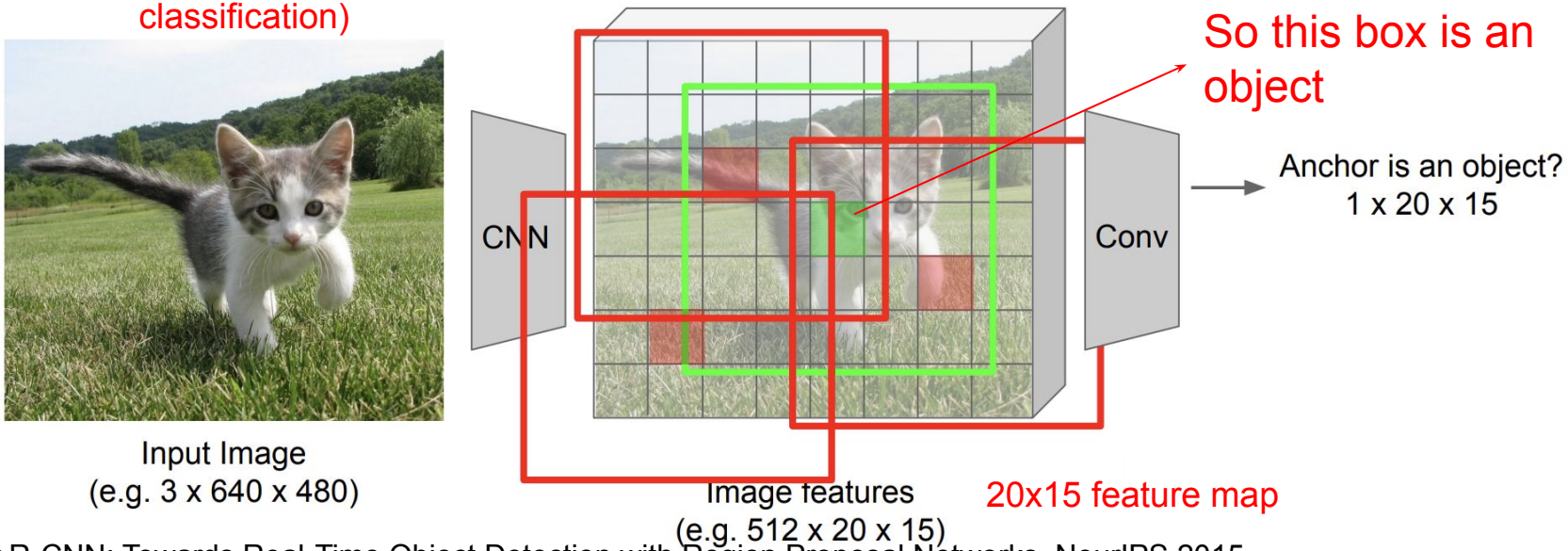
- Region Proposal Network
  - Imagine an **anchor box** of fixed size (5x5, etc.) **at each point** in the feature map



*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

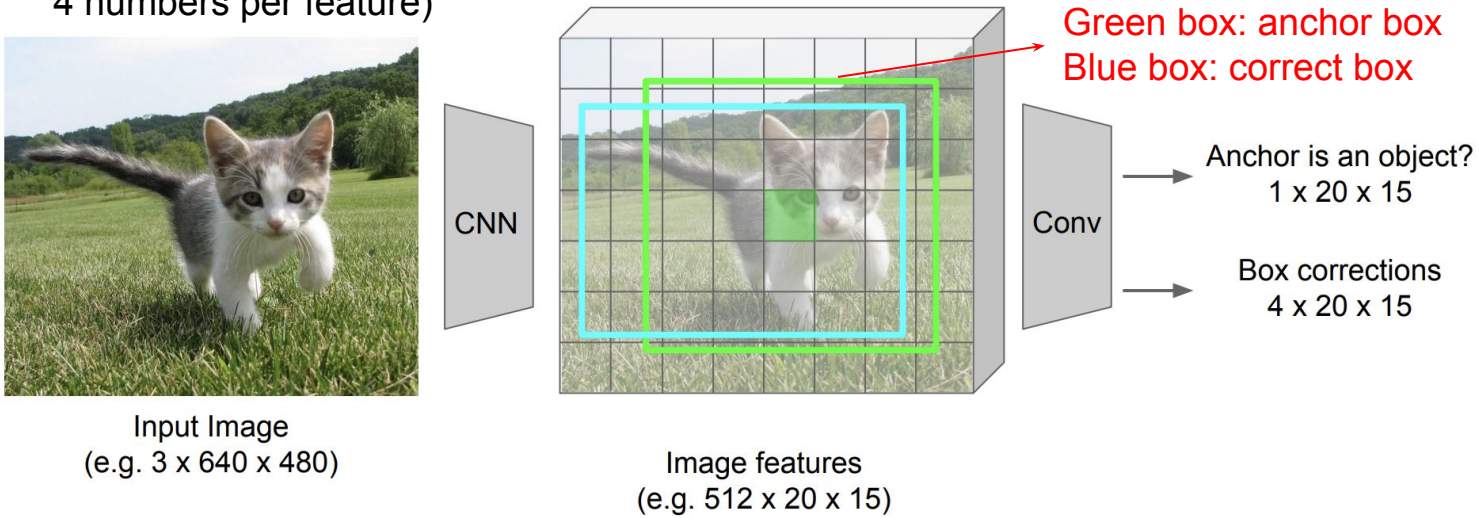
- Region Proposal Network
  - Imagine an **anchor box** of fixed size at each point in the feature map
  - At each point, predict whether the corresponding anchor contains an object (binary classification)



*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

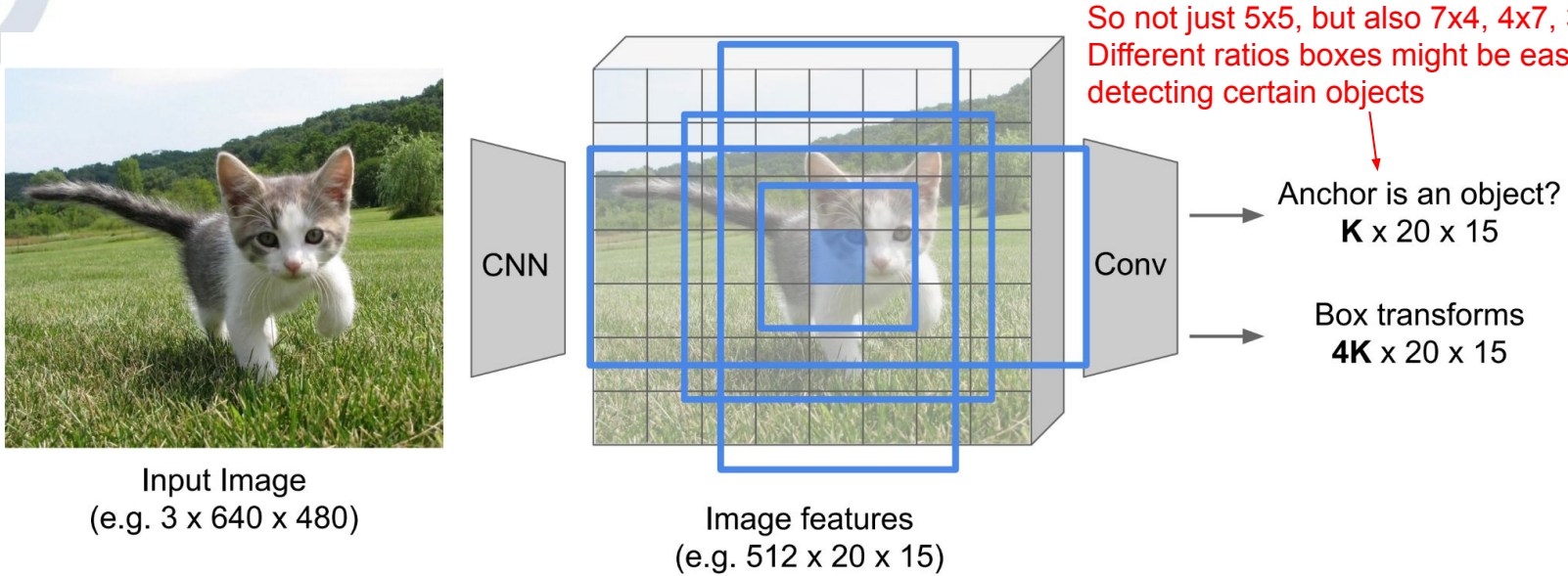
- Region Proposal Network
  - Imagine an **anchor box** of fixed size at each point in the feature map
  - For positive boxes, also predict a corrections from the anchor to the ground-truth box (regress 4 numbers per feature)



*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

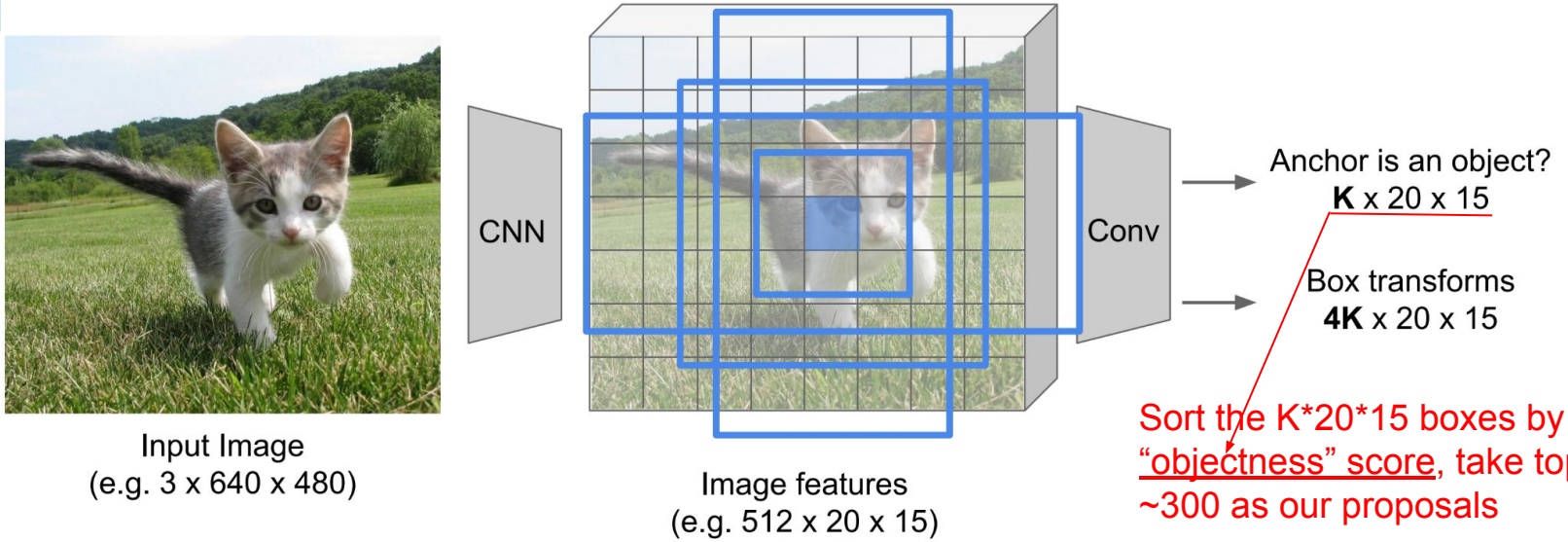
- Region Proposal Network
  - **Anchor boxes** - predefined possible **K** regular boxes at each feature location



*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

- Region Proposal Network
  - **Anchor boxes** - predefined possible **K** regular boxes at each feature location

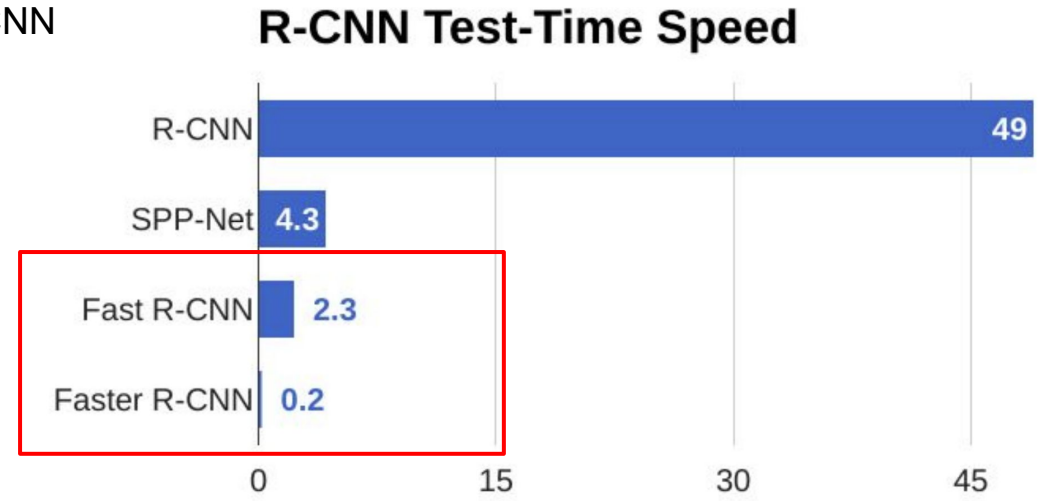


*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

- Faster R-CNN
  - Use CNN features for region proposal!
  - Called Region Proposal Network (RPN)
    - Use predefined **anchor boxes**
  - Other parts the same as Fast R-CNN

Now we save the 2 second from selective search..



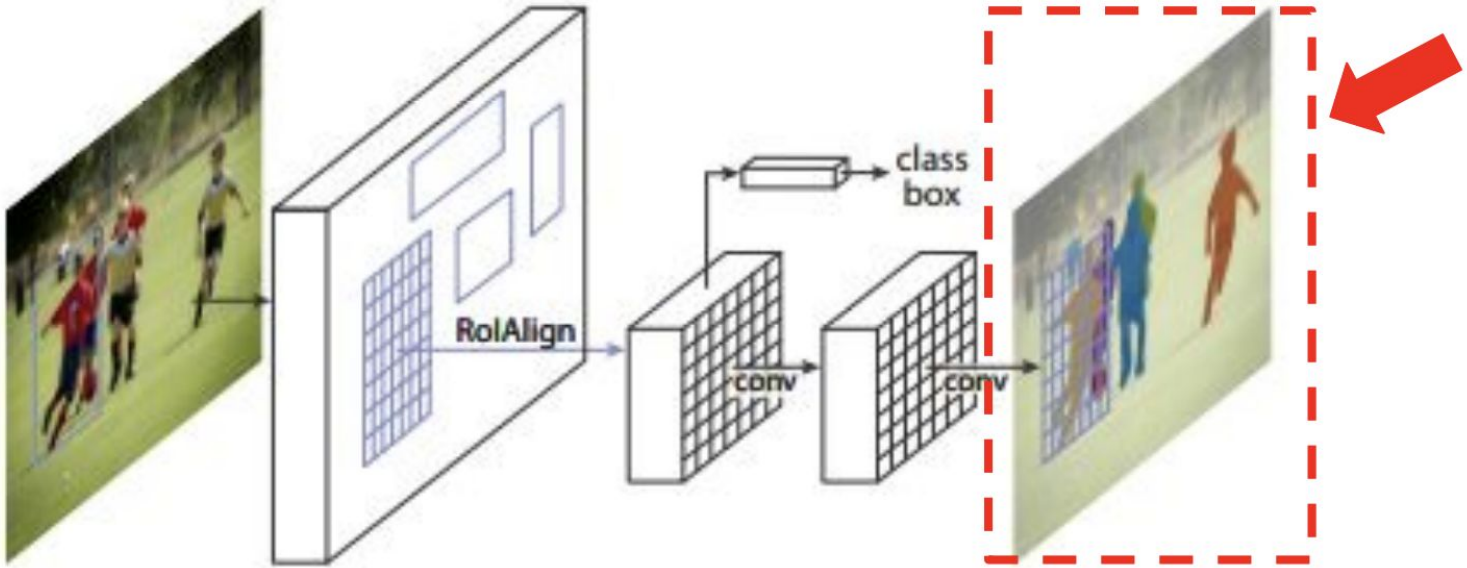
*Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS 2015

Object Detection - Two-stage method

- Mask R-CNN

So boxes are not accurate enough for objects. We need pixel-wise detections, i.e., masks

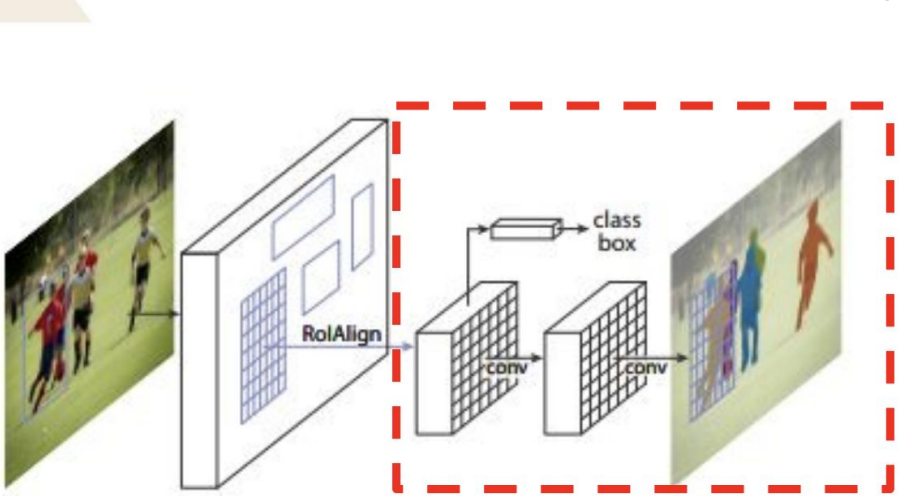
  - Mask R-CNN outputs **a binary mask** for each RoI on top of the Faster R-CNN

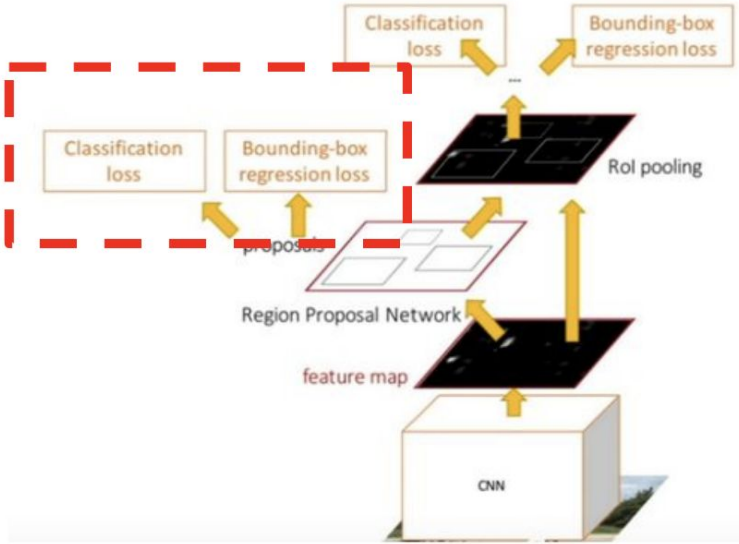


*Mask R-CNN. ICCV 2017

Object Detection - Two-stage method

- Mask R-CNN
  - Mask R-CNN adopts the same two-stage procedure with the identical RPN first stage
  - Mask R-CNN outputs a **binary mask** for each RoI for each class **in parallel to** predicting the class and box offset in the second stage





*Mask R-CNN. ICCV 2017

Object Detection - Two-stage method

- Mask R-CNN
  - Loss function
  - **The mask branch predicts encodings for K classes of each RoI** so that the network can generate masks without competition, which decouples mask and class prediction
    - So you can classify the object box first, then select that the mask for that object class

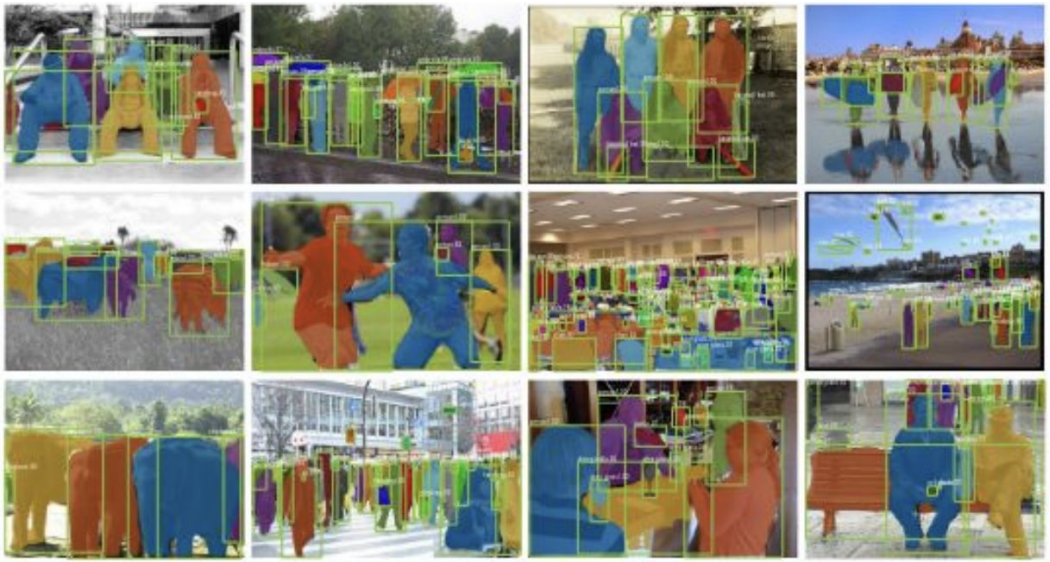
$$L = L_{cls} + L_{box} + L_{mask}$$

Why? Because the experiments show that this is the better way.

*Mask R-CNN. ICCV 2017

Object Detection - Two-stage method

- Mask R-CNN
  - Experiment - Instance segmentation



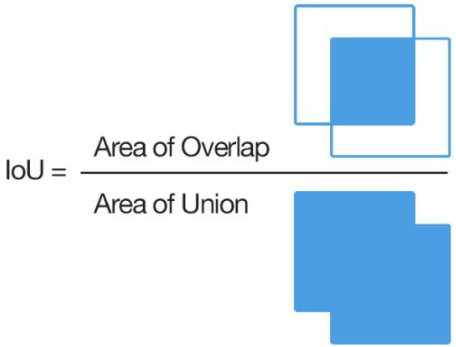
*Mask R-CNN. ICCV 2017

Object Detection - Two-stage method

- Summary
  - Two-stage
    - Region proposal - get regions-of-interest (RoIs) - Region Proposal Network (RPN)
      - Run once per image
    - Do classification and regression on the RoIs
      - Run on every RoIs
      - We can also do instance segmentation so we get a mask of the object
  - Key
    - Feature sharing
    - RoI feature cropping
  - Training - Multi-task losses
    - RPN classification and regression
    - Box classification and regression
  - **Post-processing and evaluation**
    - **Intersection-Over-Union**
    - **Non-maximum suppression**
    - **Average Precision**

Object Detection - Box Overlapping

- During the training and evaluation, we would like to determine **positives and negatives** from the object hypotheses with **ground-truth bounding boxes**
- Intersection-over-Union (IoU) is used to determine the overlapping between two bounding boxes
- IoU = 1 means 100% overlapping between two boxes, and IoU = 0 means 0% overlapping between two boxes





Object Detection - Non-maximum Suppression

- Problem: Sometimes the network predicts multiple bounding boxes for one object. How to select the best one?
  - NMS! - Basically the high score boxes will “suppress” lower score boxes they overlap. (no merging boxes)



For each class, start with the highest confidence

0. Before NMS

1. Select BBox with highest score

2. Calculate IOU

3. Remove BBox if IOU > threshold

Object Detection - Evaluation Metric - Average Precision

- COCO mAP: For COCO, AP corresponds to the average AP for IoU from 0.5 to 0.95 with a step size of 0.05. AP is the average over 10 IoU levels on 80 categories

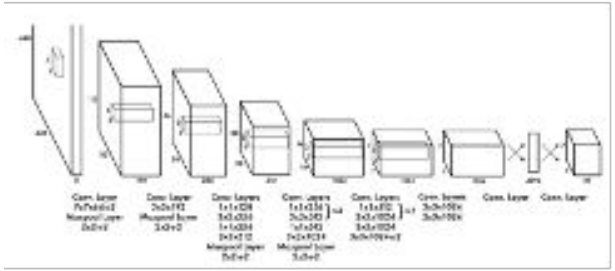
```
Average Precision (AP):
   AP                  % AP at IoU=.50:.05:.95 (primary challenge metric)
   AP^IoU=.50          % AP at IoU=.50 (PASCAL VOC metric)
   AP^IoU=.75          % AP at IoU=.75 (strict metric)
AP Across Scales:
   AP^small            % AP for small objects: area < 32^2
   AP^medium           % AP for medium objects: 32^2 < area < 96^2
   AP^large            % AP for large objects: area > 96^2
```

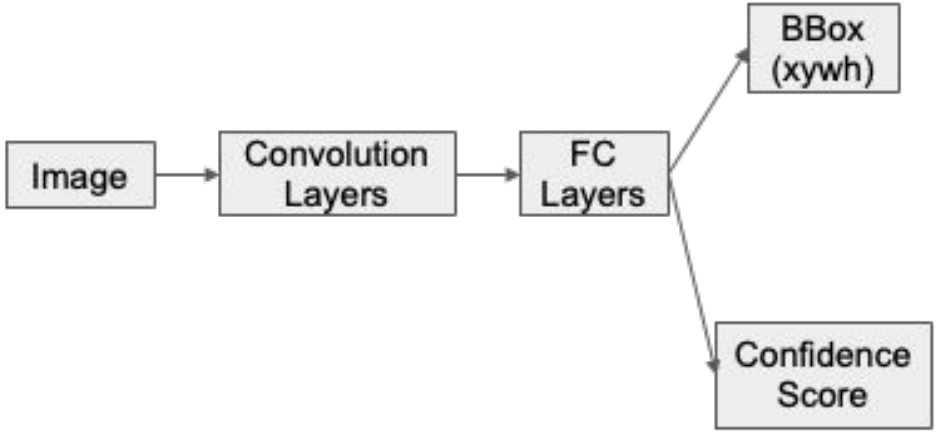
Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - **Object detection**
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

Object Detection: One-stage Model

- YOLO
  - an end-to-end (image→2d-bbox) regression problem (“unified detection”)
  - one-stage detector

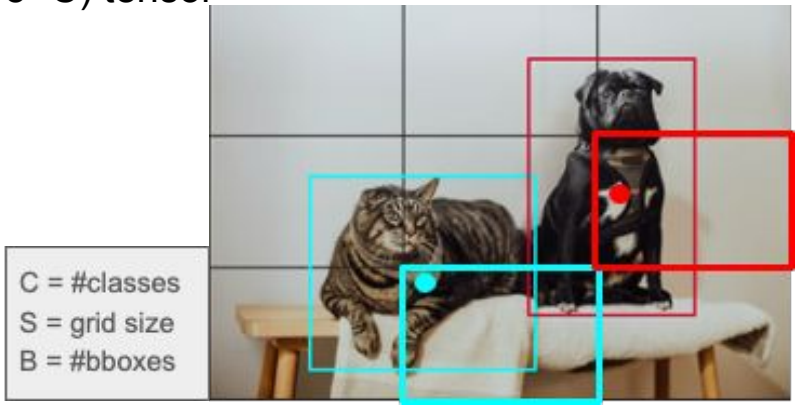




*You Only Look Once: Unified, Real-Time Object Detection. 2015

Object Detection: One-stage Model

- YOLO
  - Divide input image into an SxS grid (or Height x Width)
  - Each grid is responsible for detecting objects whose centers lie in that grid
  - Each grid predicts B bounding boxes (regression) and their confidence
  - In total, final prediction is a S x S x (B*5+C) tensor
    - [x, y, w, h, conf]



*You Only Look Once: Unified, Real-Time Object Detection. 2015

Object Detection: One-stage Model

- YOLO - Comparison
  - FPS: frame-per-second, measuring actual inference speed
  - mAP: mean average precision, higher better

		mAP	FPS
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Trading speed for mAP

*You Only Look Once: Unified, Real-Time Object Detection. 2015

Object Detection: One-stage Model

- Improvements of YOLO since 2015…
- **YOLOv2** YOLO9000:Better, Faster, Stronger. Joseph Redmon & Ali Farhad. (2017)
- **YOLOv3** YOLOv3: An Incremental Improvement. Joseph Redmon & Ali Farhad. (2018)
- **YOLOv4** YOLOv4: Optimal Speed and Accuracy of Object Detection. Alexey Bochkovski, Chien-Yao Wan & Hong-Yuan Mark Liao. (2020)
  - new features: WRC, CSP, CmBN, SAT, Mish activation, Mosaic data augmentation, CmBN, DropBlock regularization, and CIoU loss, etc.
- **YOLOv5** (GitHub)
- **YOLOv6**
- **YOLOv7, 2022 - CVPR 2023**
- **YOLOv8?? - https://github.com/ultralytics/ultralytics**
- **YOLOv9 by YOLOv4 authors in 2024**

We use YOLO11 and YOLO26

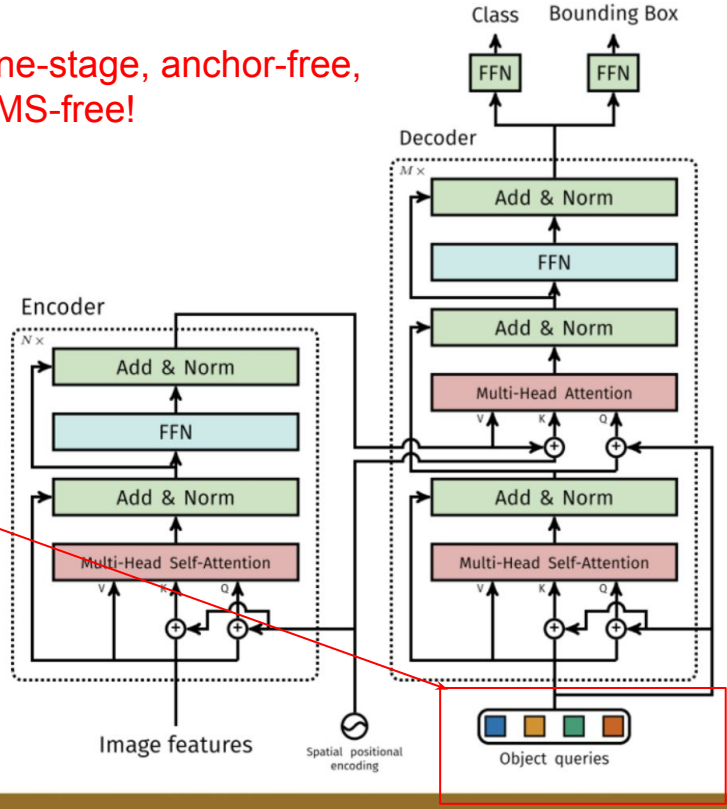
Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - **Object detection**
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

DETR: End-to-End Object Detection with Transformers

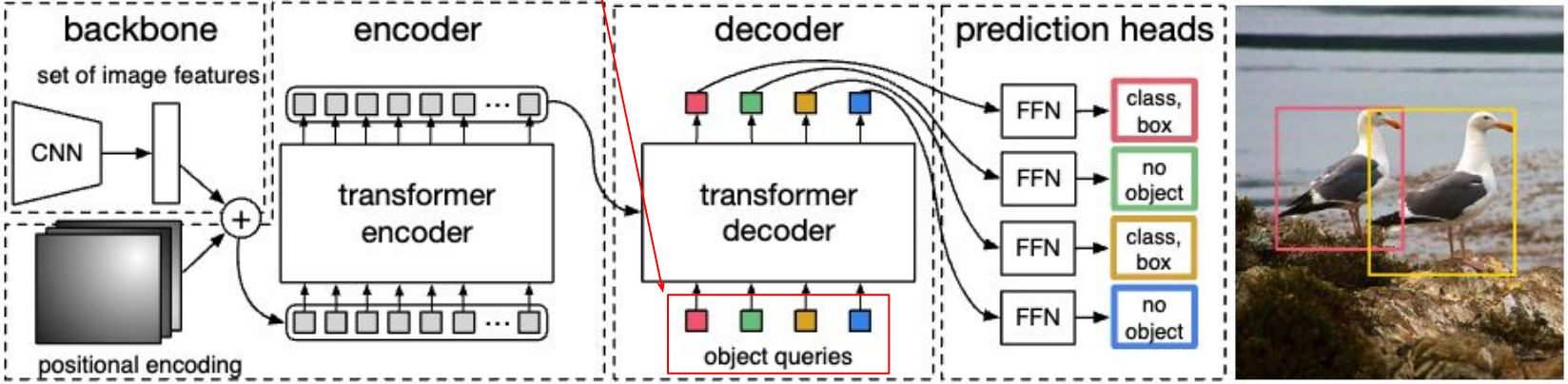
- The DEtection TRansformer (DETR) predicts all objects at once, and is trained end-to-end and **abandons the anchor boxes** and **non-maximum suppression** during testing
- DETR predicts a **fixed-size set of N predictions in a single pass**. N is chosen to be significantly larger than the typical number of objects (100 queries, etc.)
- During training, the predictions are online assigned to different ground truth boxes with a bipartite matching algorithm (Hungarian)

One-stage, anchor-free, NMS-free!



DETR: End-to-End Object Detection with Transformers

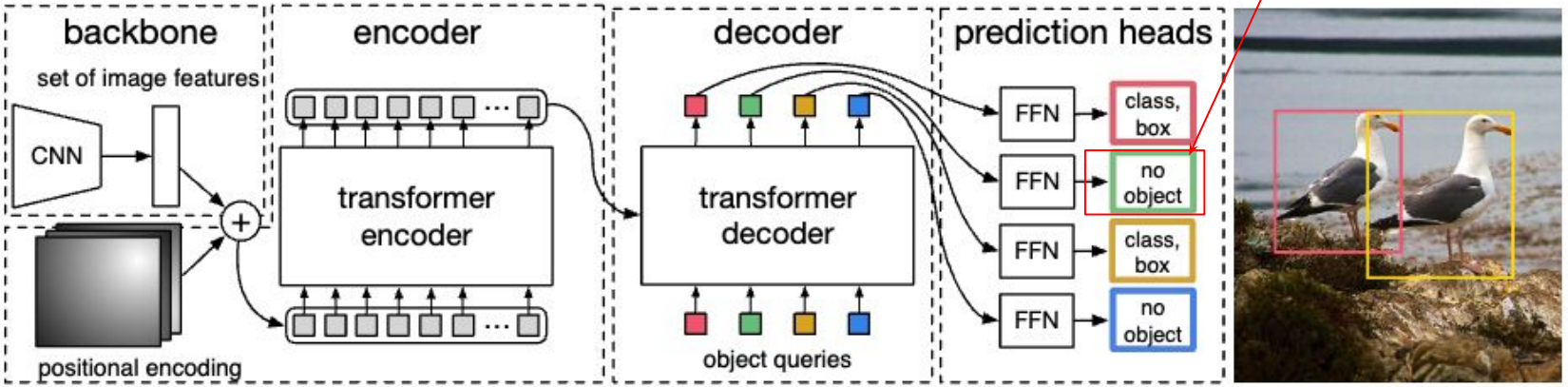
- **Backbone.** Given an input image $x_{\text{img}} \in \mathbb{R}^{H_0 \times W_0 \times 3}$, an CNN backbone obtains a visual feature map $z_0 \in \mathbb{R}^{H \times W \times C}$ (H = H0/32, W = W0/32 in the original paper), which is reshaped to HW × C feature sequence 32x downsampled
- **Transformer encoder.** The HW × C features with additive positional encoding are fed into the Transformer Encoder to conduct self-attention for multiple times
- **Transformer decoder.** It takes **N object query vectors** as inputs and decodes them **in parallel.** The query vectors are initialized randomly and learned by back-propagation



DETR: End-to-End Object Detection with Transformers

- **Prediction FFN.** The head predicts the normalized center coordinates, height and width of the box w.r.t. the input image, and the class confidences
- As N is greater than the actual number of objects, an additional **background (∅) class** is also introduced

So the fixed 100 object queries are learned “object templates”



Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - **Object detection**
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

Object Detection Practise - Running YOLOv9 on Your Laptop

- We will use the following tools
  - An open-source YOLO wrapper - ultralytics
    - https://docs.ultralytics.com/models/yolov9/
    - Should be easy to install
      - pip install ultralytics
  - Opencv - to read images from your laptop camera
    - pip install opencv-python
- The simple code base
  - https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py
  - We will do a code walkthrough

Object Detection Practise - Running YOLOv9 on Your Laptop

- A code walkthrough - imports and args

```
5  import argparse
6  import cv2
7  from ultralytics import YOLO
```

We only need three imports.
argparse for parsing some script arguments

```
parser = argparse.ArgumentParser()

parser.add_argument("--cam_num", type=int, default=0,
        help="camera num")
parser.add_argument("--model_name", default="yolov9t.pt")
```

cam_num: if your laptop connects to multiple cameras, you need to specify which. If you have an iPhone and a Mac, the phone camera is auto-connected through wifi and the laptop camera num becomes “1” not “0”
model_name: object detection checkpoint name, will automatically download if not exists

*https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py

Object Detection Practise - Running YOLOv9 on Your Laptop

- A code walkthrough - Functions

```
def run_od_on_image(
        frame_cv2, od_model,
        classes=None, conf=0.5,
        bbox_thickness=4, text_thickness=2, font_size=2):
    """
        Run object detection inference and visualize in the image.
        Return: the visualization image and the detection results
    """
```

Key function: given one image, run the detection model.
Input: the image and the model

```
results = od_model.predict(
    frame_cv2,
    classes=classes,  # you can specify the classes you want
    # see here for coco class indexes [0-79], 0 is person: h
    #classes=[0, 32], # detect person and sports ball only
    conf=conf)
```

By default it will return all 80 COCO classes' detection results.
conf: the confidence score threshold. You want more results, make conf=0.1, etc.

*https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py

Object Detection Practise - Running YOLOv9 on Your Laptop

- A code walkthrough - Functions

```python
def count_people(det_results):
    """
        Given the detection result, count the number of people
    """
    num_people = 0
    result = det_results[0] # we run it on single image
    for box in result.boxes:
        if int(box.cls[0]) == 0:
            num_people += 1
    return num_people
```

An example function: take the detection results, and count the number of people.

The detection results contain a list of boxes

If you want to get bbox coordinates: `box.xyxy[0]`

Note there are xyxy and xywh to represent bbox

The name of the object class: `result.names[int(box.cls[0])]`

*https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py

Object Detection Practise - Running YOLOv9 on Your Laptop

- A code walkthrough - Main script - initialization

```
# initialize the object detection model
# this will auto download the YOLOv9 checkpoint
# see here for all the available models: https:/
model = YOLO(args.model_name)
```

This calls the API and build a model (download and load the model weights)

```
cam = cv2.VideoCapture(cam_num)

try:
    if cam is None or not cam.isOpened():
        raise Exception("failed to grab camera %s" % cam_num)
```

Try a smaller model if you feel the frame rate is too low

Opencv is a convenient (and actually almost the fastest) tool to process media files.

We can open a video stream from the camera

*https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py

Object Detection Practise - Running YOLOv9 on Your Laptop

- A code walkthrough - Main script - main loop - Usually we process video stream indefinitely

```
while True:
    ret, frame = cam.read()
    if not ret:
        raise Exception("Error: Could not read frame from webcam.")

    frame, det_results = run_od_on_image(frame, model)

    num_people = count_people(det_results)

    frame = cv2.rectangle(
        frame, (0, frame.shape[0]-20),
        (800, frame.shape[0]-120), (0, 0, 0), -1)
    frame = cv2.putText(
        frame, "# People: %d" % num_people,
        (20, frame.shape[0]-40), cv2.FONT_HERSHEY_SIMPLEX,
        fontScale=3, color=(0, 255, 0), thickness=8)

    cv2.imshow("frame", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

The main loop:

We process video stream frame-by-frame in a loop until “Q” is pressed

For each frame, we run object detection and visualization, and then count the number of people. You can add your customized analysis here.

*https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py

Object Detection Practise - Running YOLOv9 on Your Laptop

- Try it yourself
  - YOLOv9
  - YOLO-World
    - Open-vocabulary object detector*
    - $ python run_open_od_on_camera.py --prompts "blue box" --prompts "yellow box"

*https://github.com/JunweiLiang/tennis_project/blob/main/run_od_on_camera.py
*https://github.com/JunweiLiang/tennis_project/blob/main/run_open_od_on_camera.py
*YOLO-World: Real-Time Open-Vocabulary Object Detection. CVPR 2024

Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - **Object detection**
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

Visual Tracking

- Problem
  - Tracking the same objects/features across multiple video frames
- Types
  - **Tracking without detection**
    - Manually given a bounding box, then track the box
    - Single Object Tracking (SOT, a.k.a. VOT)
      - Traditional vision method based and deep learning based (SiamRPN, CVPR 2019)
      - Not very useful (See DJI active track)
  - **Tracking-by-detection**
    - Multi-Object Tracking (MOT)
      - Online tracking & offline tracking
      - Single camera
      - Multi-camera

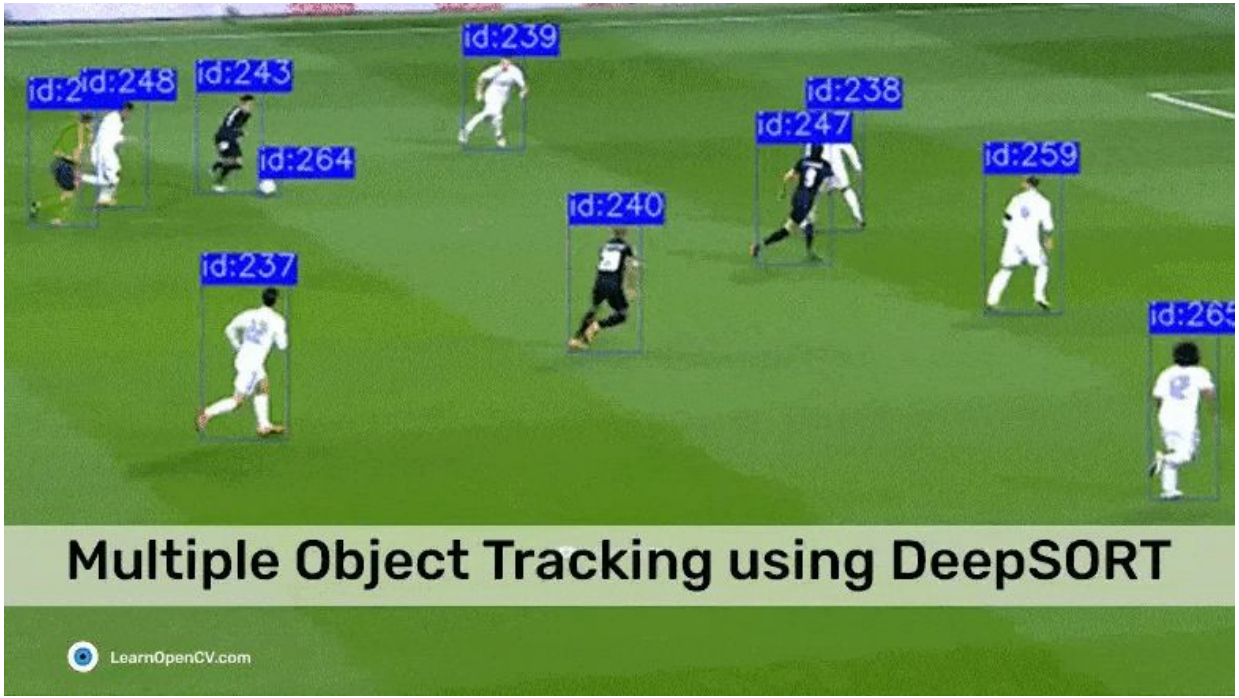
SOT Example

- How single object tracking/tracking without detection is used in real-world application
  - DJI Active Track / Quick shot
  - Click a point to track



MOT Examples

- Track players in soccer



MOT Examples

- Track players in soccer

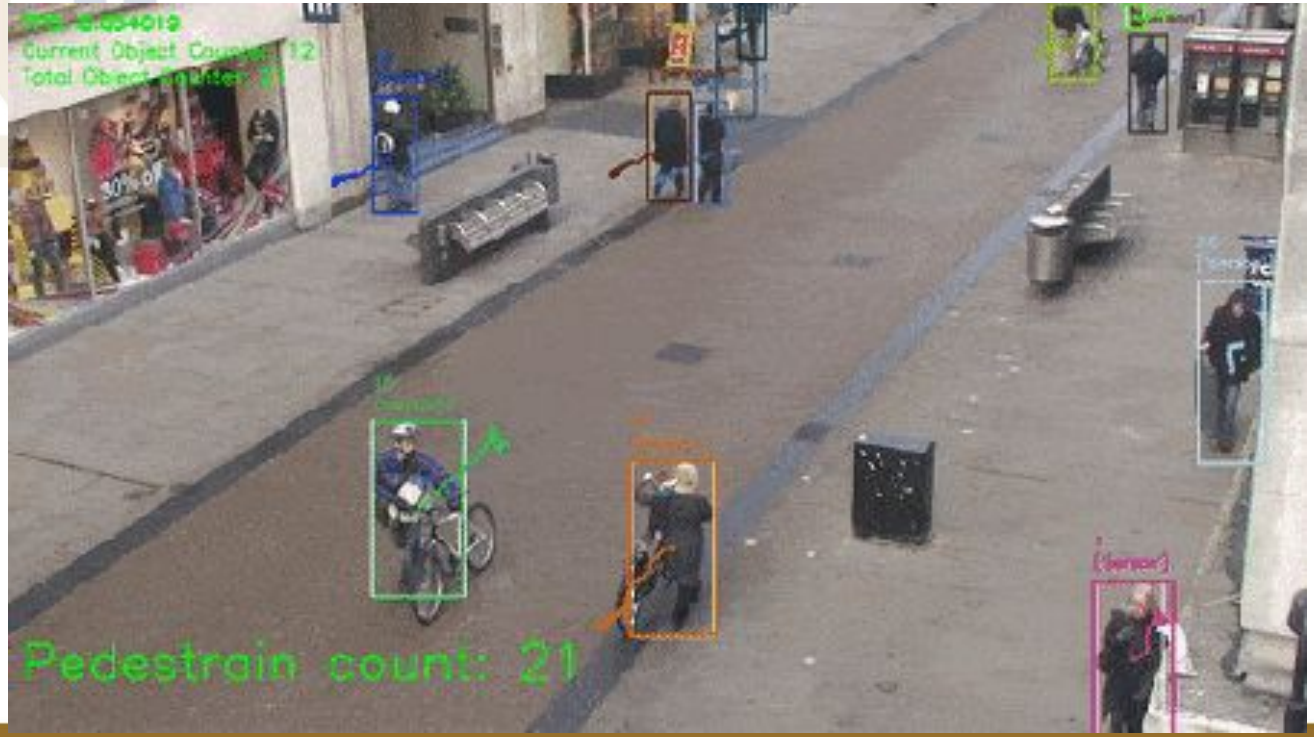


The statistics of each player can then be automatically summarized.

For example, the distance covered.

MOT Examples

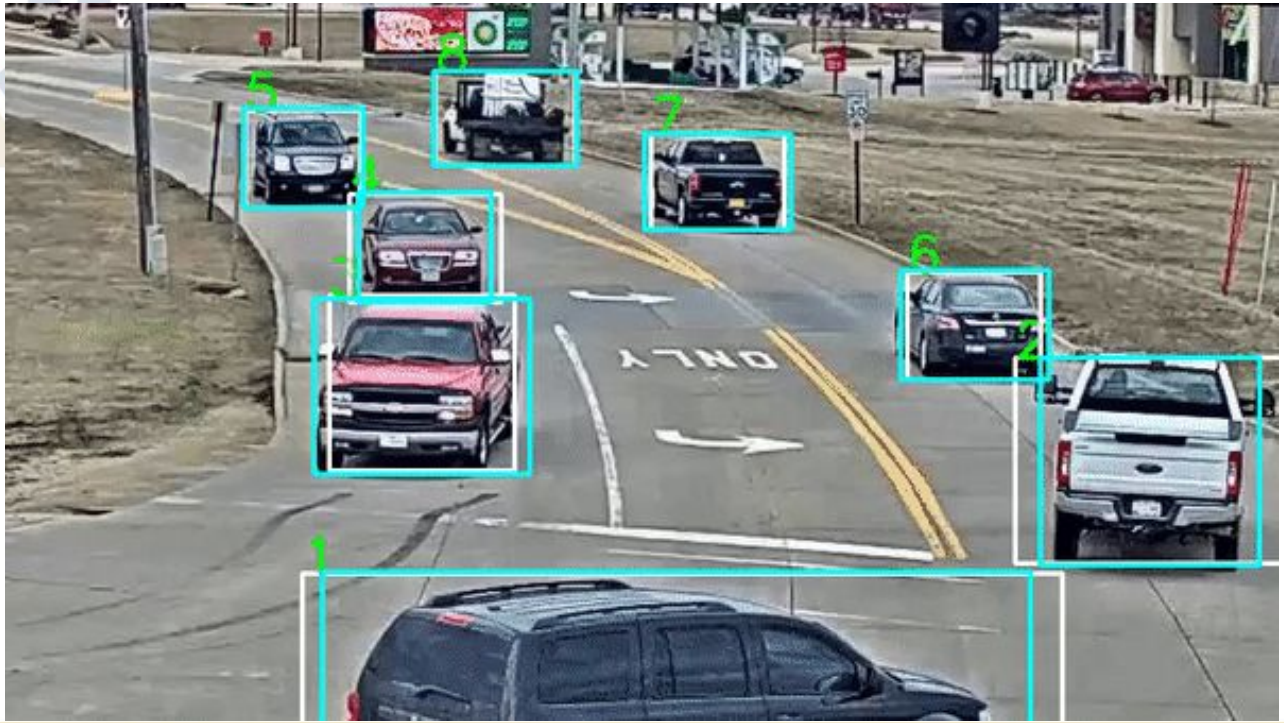
- Person counting from a camera



You can also count how many pedestrians have passed your camera in a certain time frame

MOT Examples

- Vehicle tracking



Tracking is widely used in traffic cameras.

Compared to person tracking, vehicles are easier and with less diverse background.

Tracking-by-Detection

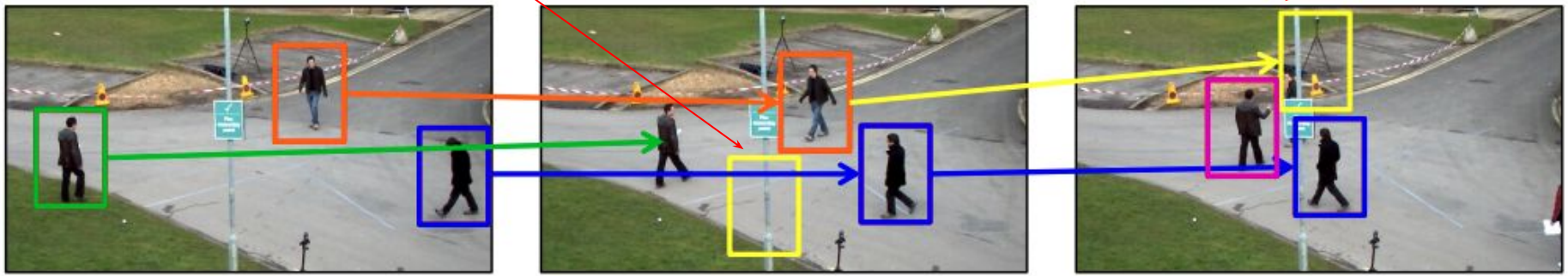
- Challenges
  - Variations due to geometric changes (pose, scale)
  - Variations due to photometric factors (change of lighting conditions, appearance)
  - **Occlusions**
  - Non-linear motion
  - Limited resolution, blurry, far away (so even detections might be missing)
  - Appearance is often very similar

Tracking-by-Detection

- Problem
  - Input: object boxes
  - Objective: estimate **object state** over time
    - **State**: box position. This is a naming convention from robotics
- Key ingredients
  - Detection
  - Tracking
    - **State estimation**
      - predict where the boxes might be
    - **Data association**
      - Similarity measurement

Tracking-by-Detection

- Problem
  - Input: target boxes
  - Objective: estimate target state over time (given the a bunch of detected boxes across time)
  - State: box position
- First thing to note
  - Detections are not perfect!

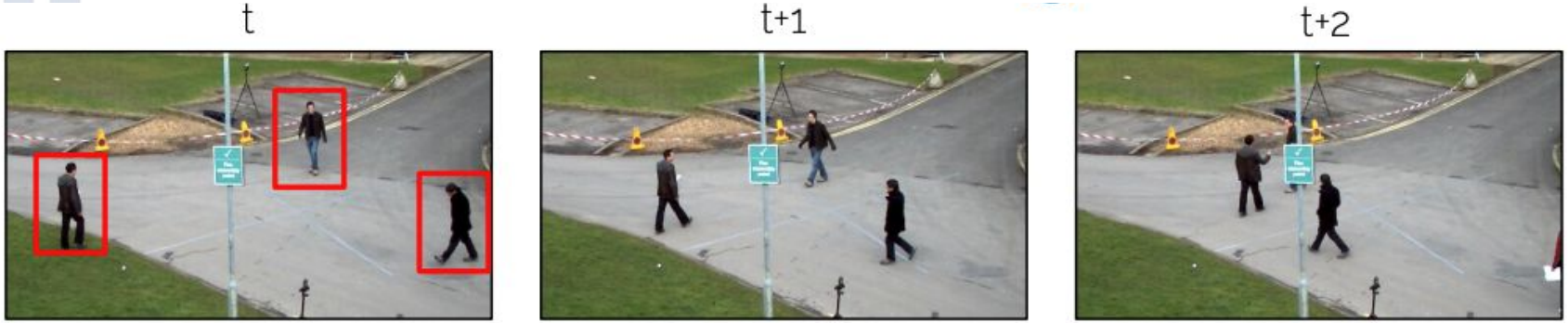


Find detections that match and form a trajectory

Online vs Offline Tracking

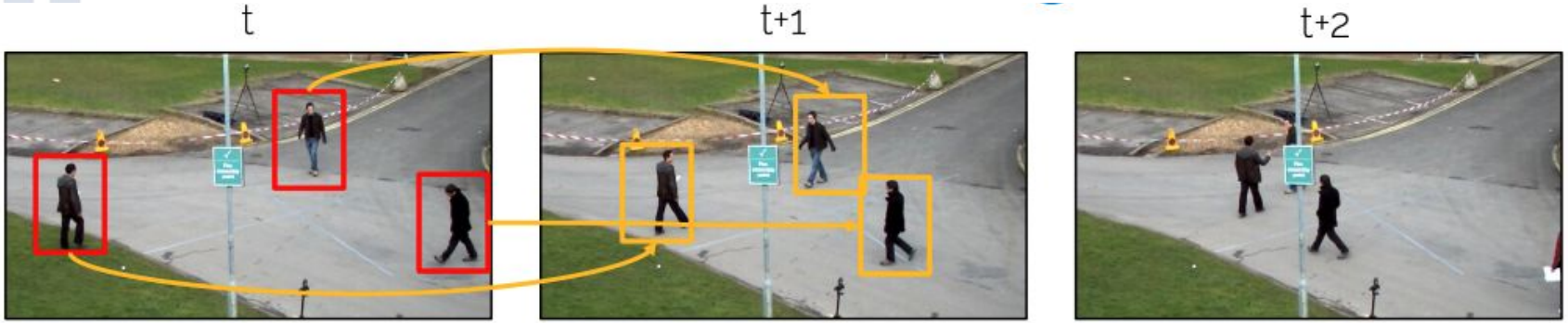
- Online tracking
  - Processes two frames at a time
  - For real-time applications
  - Prone to drifting
    - Hard to recover from errors or occlusions
- Offline tracking
  - Processes a batch of frames
  - Good to recover from occlusions
  - Not suitable for real-time applications
- We will only cover online tracking method in this class
  - As this is the most used method in real-world applications like surveillance and robots

Online Tracking - Example Steps



- 1. Track initialization (e.g. using a object detector) 

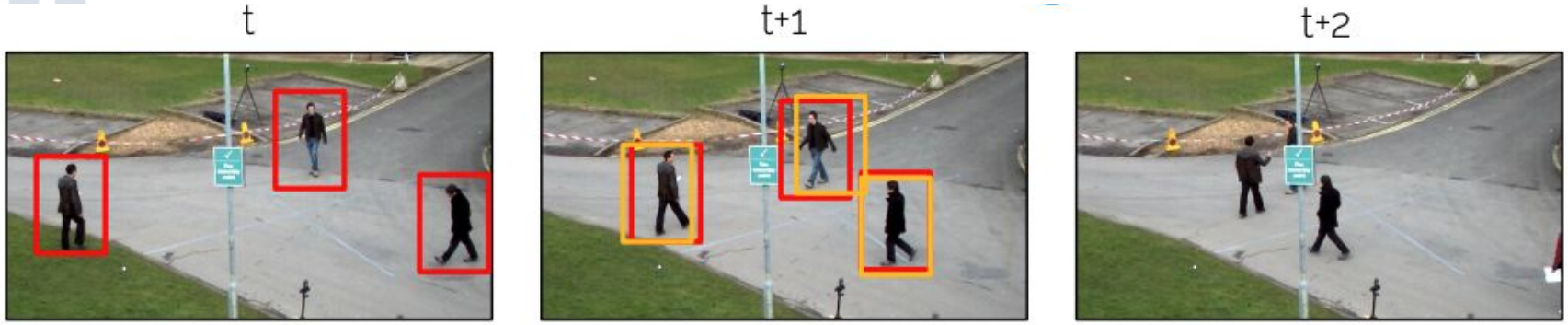
Online Tracking - Example Steps



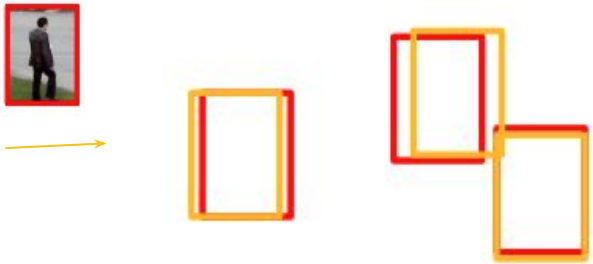
- 1. Track initialization (e.g. using a object detector) 
- 2. Prediction of the next position

For example, we can predict where the track's box will be by taking into its previous moving trajectory

Online Tracking - Example Steps



- 1. Track initialization (e.g. using a object detector)
- 2. Prediction of the next position
- 3. Matching predictions with detections
  - Data association based on appearance, location, etc.



Prediction of the next position

State, Kalman filter, these are common in robotics. We will see “state” again in later lectures

- **Kalman filter**
  - **State** space of a object box xywh $(u, v, \gamma, h, \dot{x}, \dot{y}, \dot{\gamma}, \dot{h})$
    - 4 dim box coordinates
    - 4 dim their respective velocity
    - The Kalman filter for each track will have 8 scalars for the mean of these variables, and a 8x8 matrix of their covariance (which means it assumes a multivariate gaussians)

This is assumed to be changing over time

  - Given 4 dim box coordinates as a new observation of the current timestep
    - Update the track’s current KF states (8 mean and 8x8 covar)
      - Based on previous states and **some calculation**
        - This involves some computation of Kalman gain, you can check the code below. Not going into this detail today
    - Just consider it as **a linear model that extrapolate tracks** (object states) assuming constant velocity

*Full code: https://github.com/JunweiLiang/Object_Detection_Tracking/blob/master/deep_sort/kalman_filter.py#L157

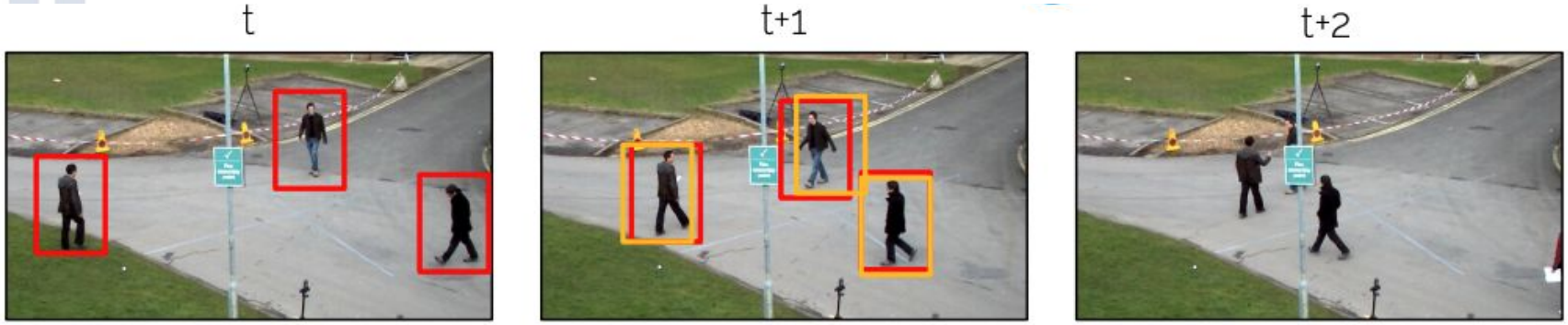
*Simple Online and Realtime Tracking with a Deep Association Metric, 2017

Comparing detected boxes and predicted boxes

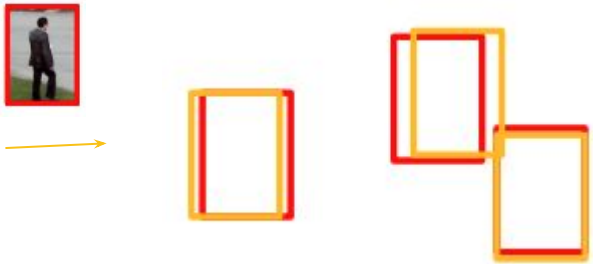
- Blue box: detected boxes
- White box: predicted boxes
- See the white box slowly enlarges



Online Tracking - Example Steps



- 1. Track initialization (e.g. using a object detector)
- 2. Prediction of the next position (motion model)
- 3. **Matching predictions with detections**
  - **Data association** based on appearance, location, etc.

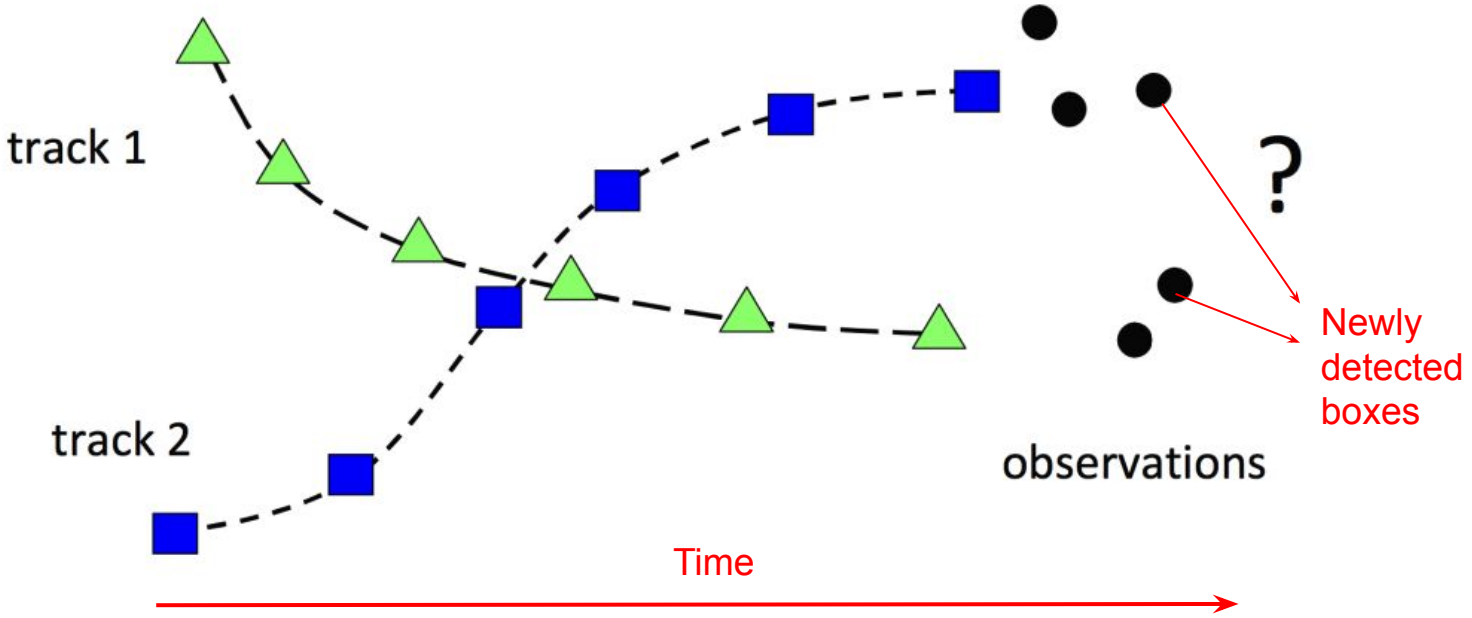


Data Association

- Given each estimated track state (potential target box locations)
  - Assign tracks with new boxes (new observations)
  - Which is a discrete combinatorial optimization problem

Data Association

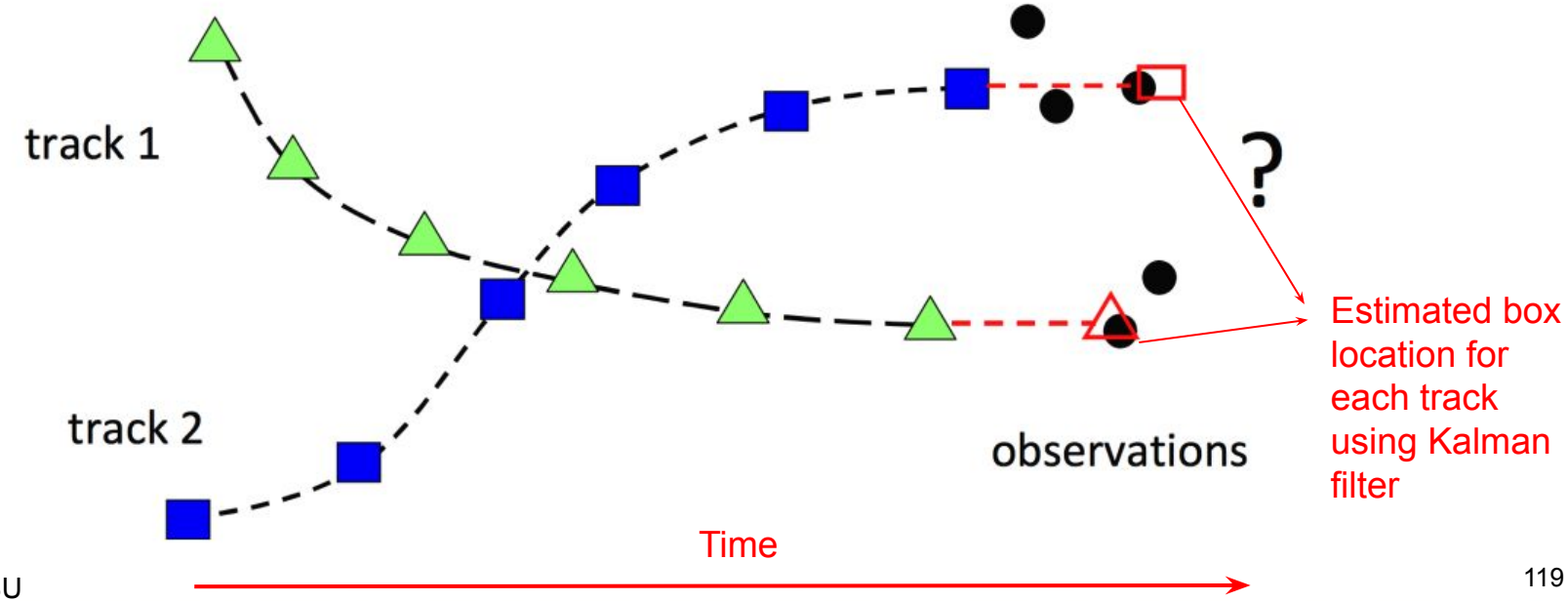
- Given each estimated track state (potential target box locations)
  -



Credit: Collins, PSU

Data Association

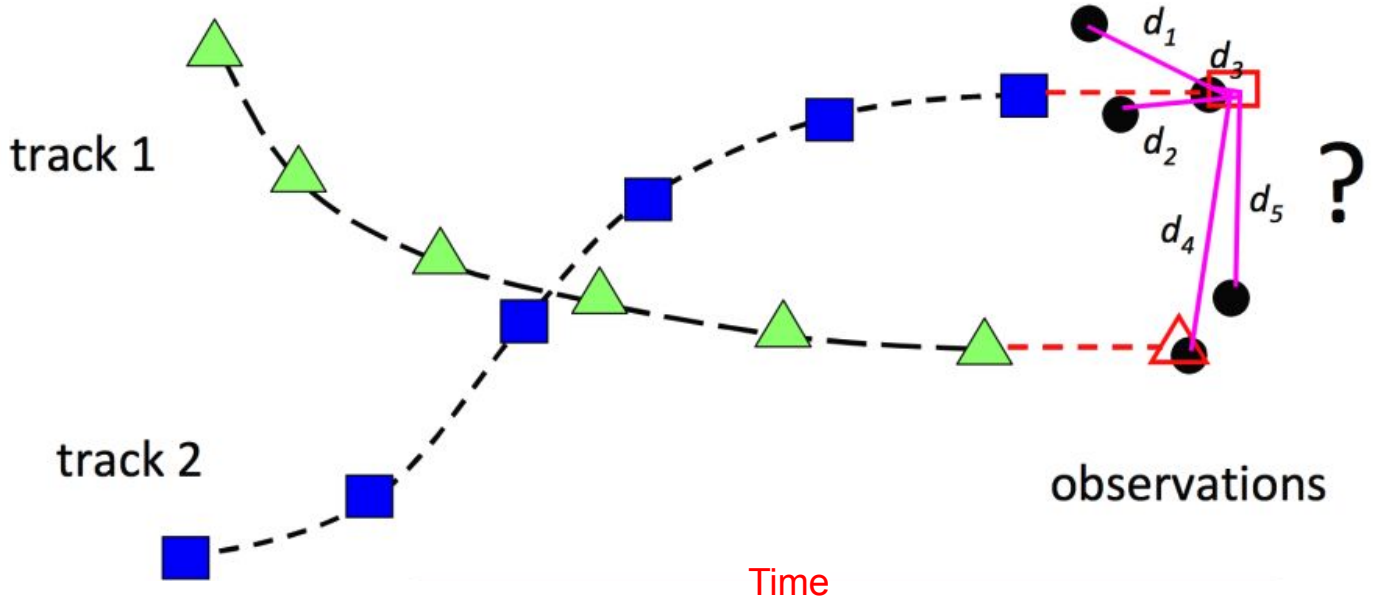
- Given each estimated track state (potential target box locations)
  -



Credit: Collins, PSU

Data Association

- Given each estimated track state (potential target box locations)
  -

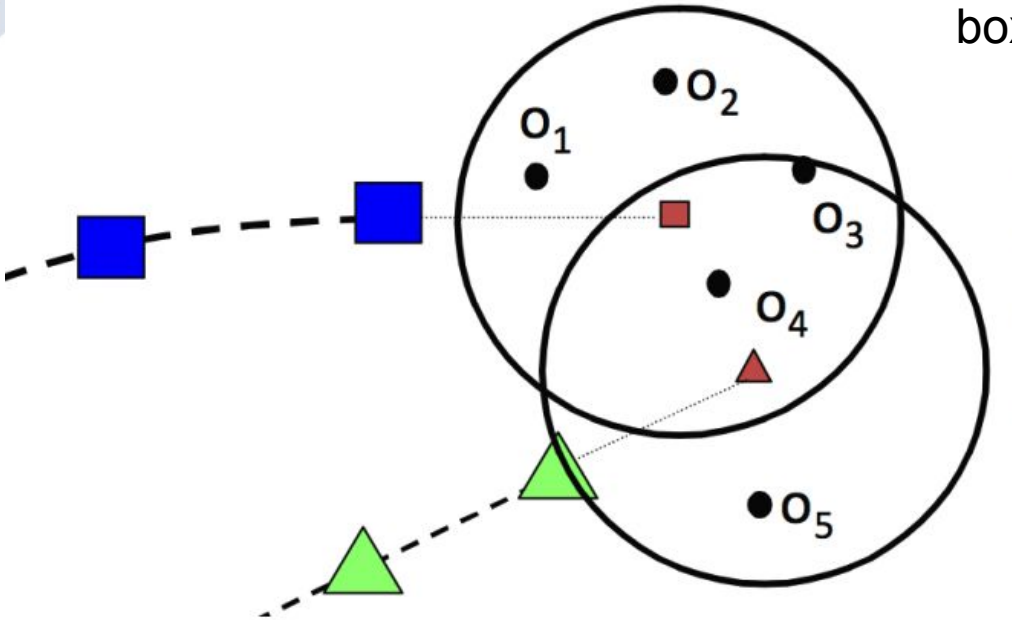


How to assign estimated box to new detected box?

Credit: Collins, PSU

Data Association

- Given each estimated track state (potential target box locations)



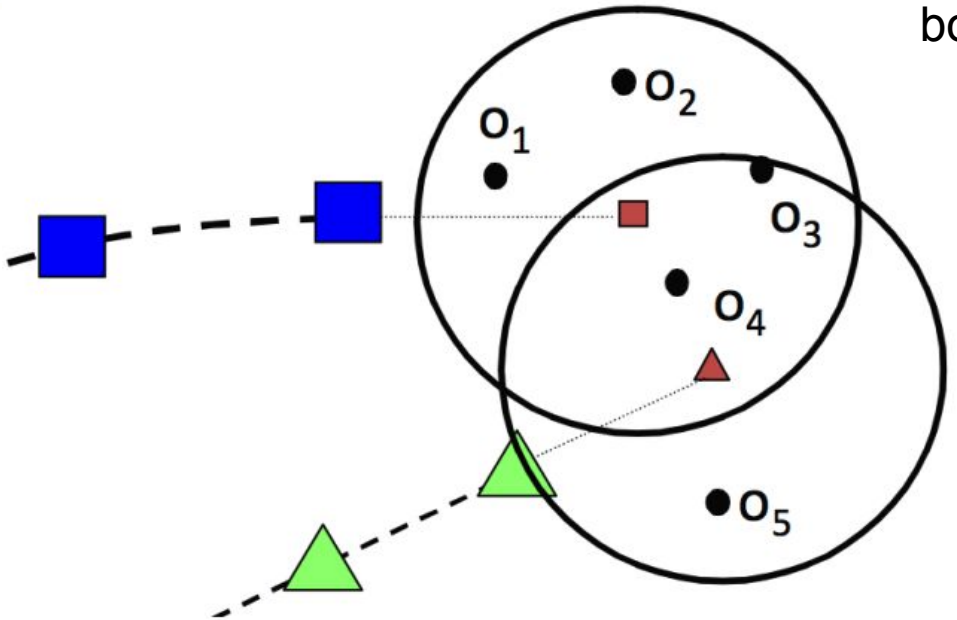
New boxes	Track 1	Track 2
1	3.0	
2	5.0	
3	6.0	1.0
4	9.0	8.0
5		3.0

We have 5 new observed boxes

Suppose these are the matching scores (closer the higher, etc.)

Data Association

- Given each estimated track state (potential target box locations)



New boxes	Track 1	Track 2
1	3.0	
2	5.0	
3	6.0	1.0
4	9.0	8.0
5		3.0

We can only do 1-to-1 assignment.

Using greedy method, Track 2 will be assigned to box 5

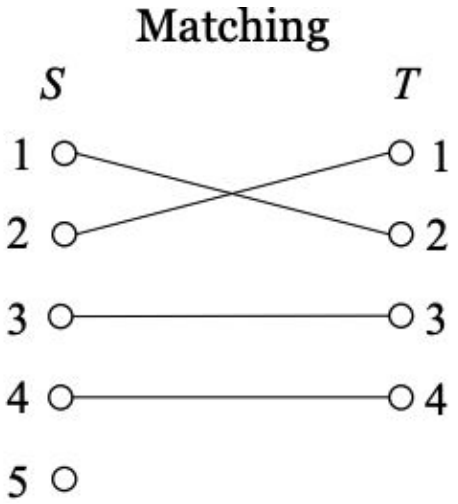
Non-optimal!

Total score is not maximized

Data Association

We skip the algorithm detail. You just need to know what it is for and when to use it. Tools in scipy compute for you.

- **Hungarian Algorithm**
  - For assignment problem, also called **weighted bipartite matching problem**
    - Bipartite graph
      - Has two sets of **nodes** S, T;
        - V = union(S, T)
      - And a set of **edges** E connecting them
      - A matching on a bipartite graph is to find E that no two edges are connected to the same node
      - So some nodes can be left out
  - Complexity - **<u>guaranteed optimal solution</u>**
    - O(n^3), where n is the number of elements in each set.



Data Association

- Example result - matching 5 nodes with other 5 nodes

Given the similarity matrix

	1	2	3	4	5
1	0.95	0.76	0.62	0.41	0.06
2	0.23	0.46	0.79	0.94	0.35
3	0.61	0.02	0.92	0.92	0.81
4	0.49	0.82	0.74	0.41	0.01
5	0.89	0.44	0.18	0.89	0.14

Greedy Solution

Score=3.77

Go from row 1 to row 5, select the one with the highest score

	1	2	3	4	5
1	0.95	0.76	0.62	0.41	0.06
2	0.23	0.46	0.79	0.94	0.35
3	0.61	0.02	0.92	0.92	0.81
4	0.49	0.82	0.74	0.41	0.01
5	0.89	0.44	0.18	0.89	0.14

Optimal Solution

Score=4.26

Hungarian algorithm (An iterative process that adjusts the matching)
Avoids a bad match

Data Association

- Hungarian algorithm
  - Implementation
    - Given a cost_matrix of n-by-m, get the assignment indices

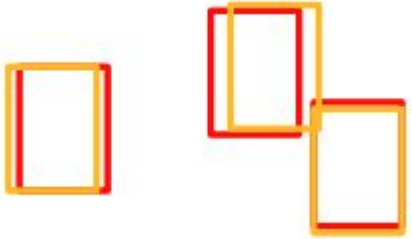
```
from scipy.optimize import linear_sum_assignment
indices = linear_sum_assignment(cost_matrix)
```

Note here it uses cost matrix (lower the better) instead of similarity matrix (higher the better)

- Documentation: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html
- Example use in tracking assignment:
  https://github.com/JunweiLiang/Object_Detection_Tracking/blob/master/deep_sort/linear_assignment.py#L12

Online Tracking - How is the cost matrix computed?





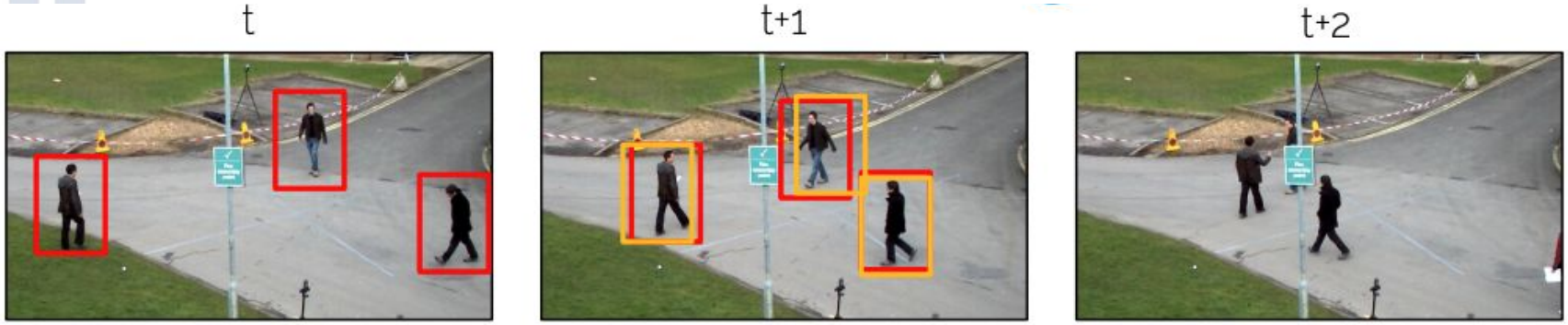


- 3. Matching predictions with detections
  - Data association based on appearance, **location**, etc.

Compute IOU between predicted boxes and observed boxes



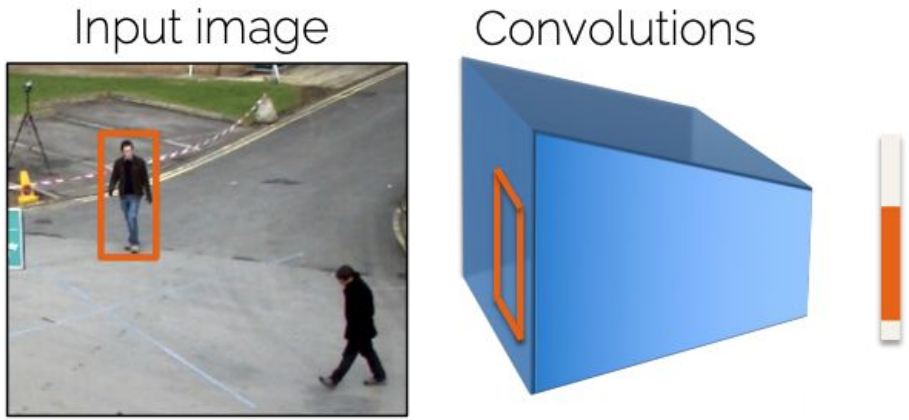
Online Tracking - How is the cost matrix computed?



- 3. Matching predictions with detections
  - Data association based on **appearance**, location, etc.

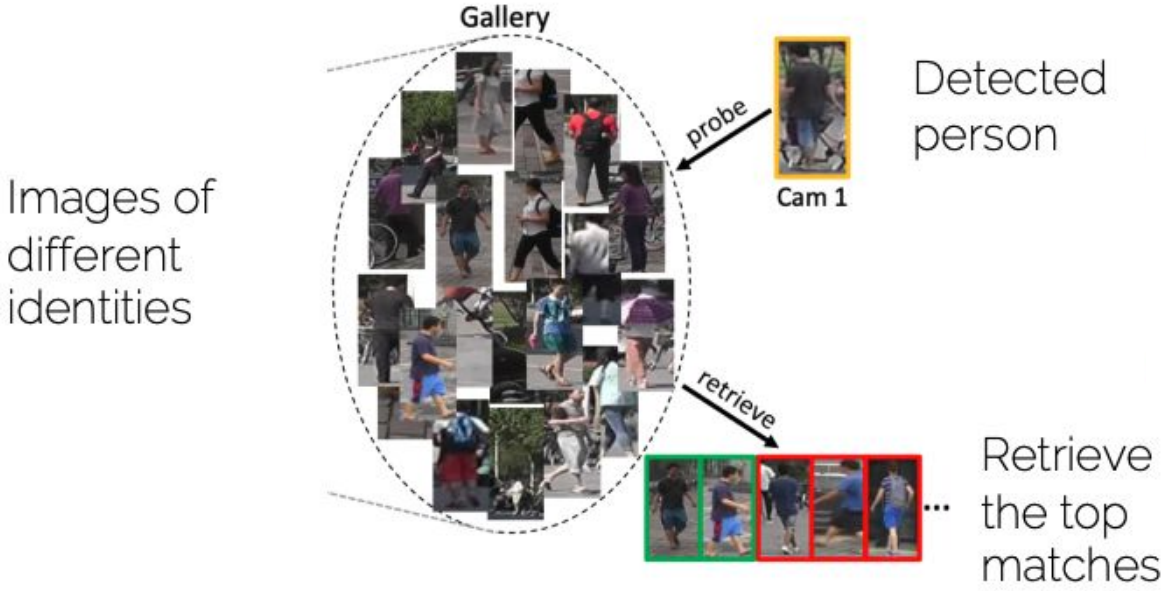
Appearance Feature for matching

- Matching predictions with detections
  - Data association based on **appearance**, location, etc.
  - We can use **ROIAlign** to get feature vector of the object from object detection model - Save computation!
    - Then we can compute cosine distance between two vectors



Better Appearance Feature - Person Re-Identification

- Use **metric learning/similarity learning/contrastive learning** to get better discriminative person features
  - Person Re-ID is an active area of research for many years



Online Tracking - Summary

- 1. Track initialization (e.g. using a object detector)
  - YOLO, Faster RCNN, etc.
- 2. Prediction of the next position
  - Kalman filter
- 3. Matching predictions with detections
  - Data association based on appearance, location, etc.

Upgrade your detector and you will get a significant boost in performance

Super fast

Hungarian Algorithm

The SORT method: Simple Online and Realtime Tracking, 2016

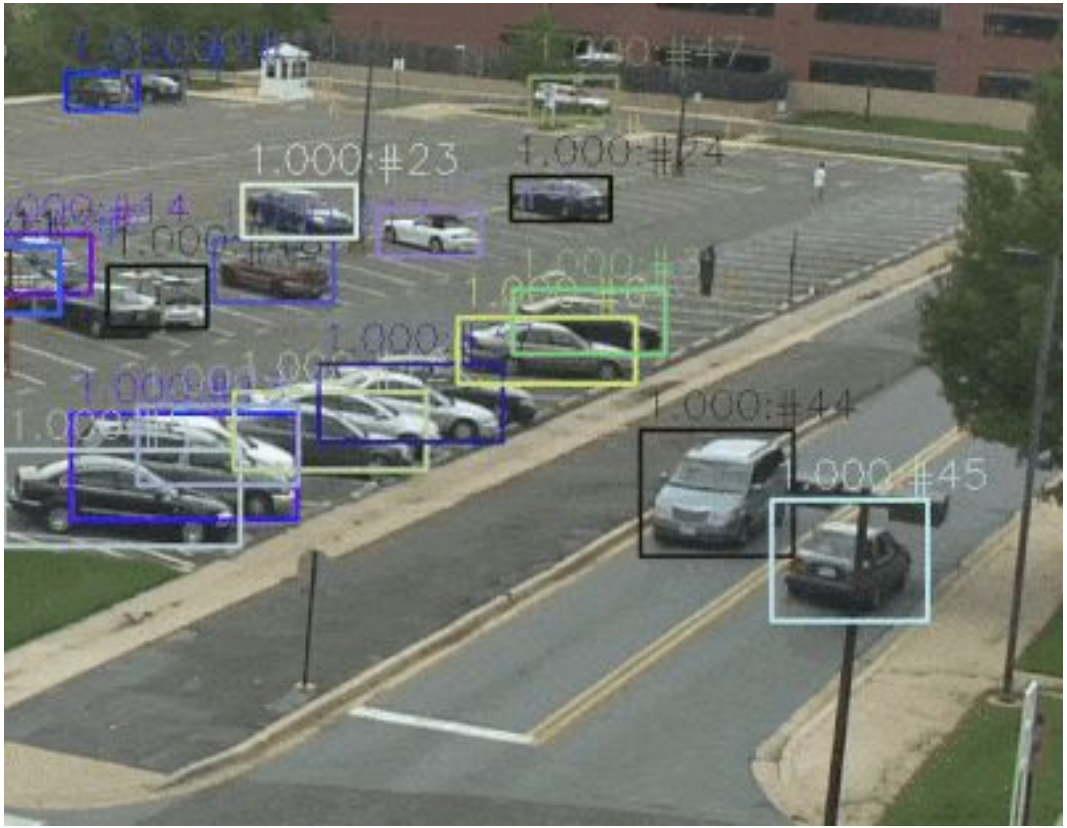
The DeepSORT method: Simple Online and Realtime Tracking with a Deep Association Metric, 2017

DeepSORT is still on par with SOTA (Bytetrack, ECCV 2022) given the same object detection model

Full object detect and track code example:
https://github.com/JunweiLiang/Object_Detection_Tracking/blob/master/obj_detect_tracking_multi_queuer_tmot.py#L490

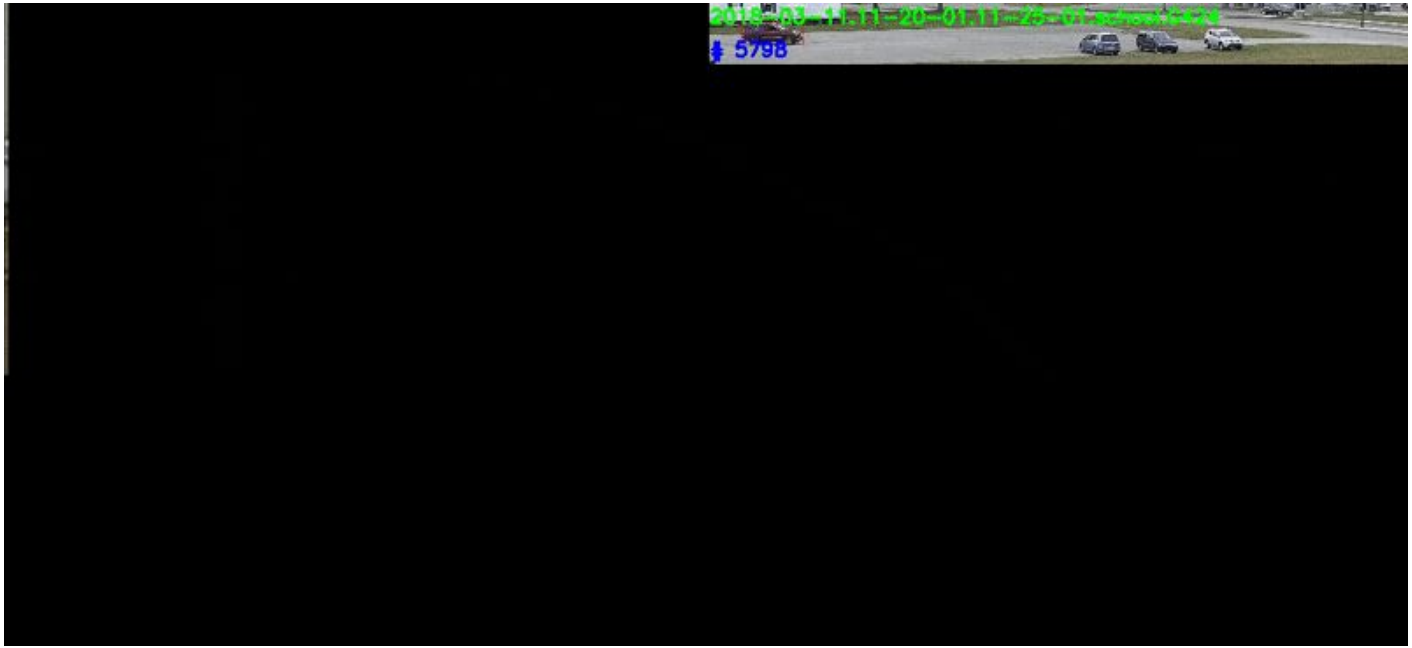
Tracking-by-Detection - Single Camera MOT

- Example
  - Motion
  - Change in scale
  - Occlusion
  - See the missing few boxes of the parked cars but the ID stays the same



Tracking-by-Detection - Multi-Camera MOT

- Example
  - Need to perform ReID on multiple cameras



Agenda

- Objectives
  - We will learn how to detect and track objects in images and videos this week
- The tasks
  - Logistics of P1 and P2: Paper presentation and EgoObject Kaggle Task
    - Today we have In-class Quiz 1 from 10:10-10:25
  - **Object detection**
    - Two stage model
    - One stage model
    - Transformer model
  - Object detection practise
  - Multi-Object Tracking (MOT)

In-Class Quiz 1

- Link
  - https://hkust-gz.instructure.com/courses/3918/assignments/28723
  - Password:

- Rules
  - 10:10 - 10:25
  - You are not allowed to use ChatGPT and alike

Backup Content - Long-term MOT with trajectory prediction

- Objectives
  - We have learned about object detection. Now we will learn about how to represent them in videos (by giving them IDs)
- The tasks
  - Object detection practise
  - Multi-Object Tracking (MOT)
  - **Long-term MOT with trajectory prediction**

We can combine trajectory prediction with MOT…

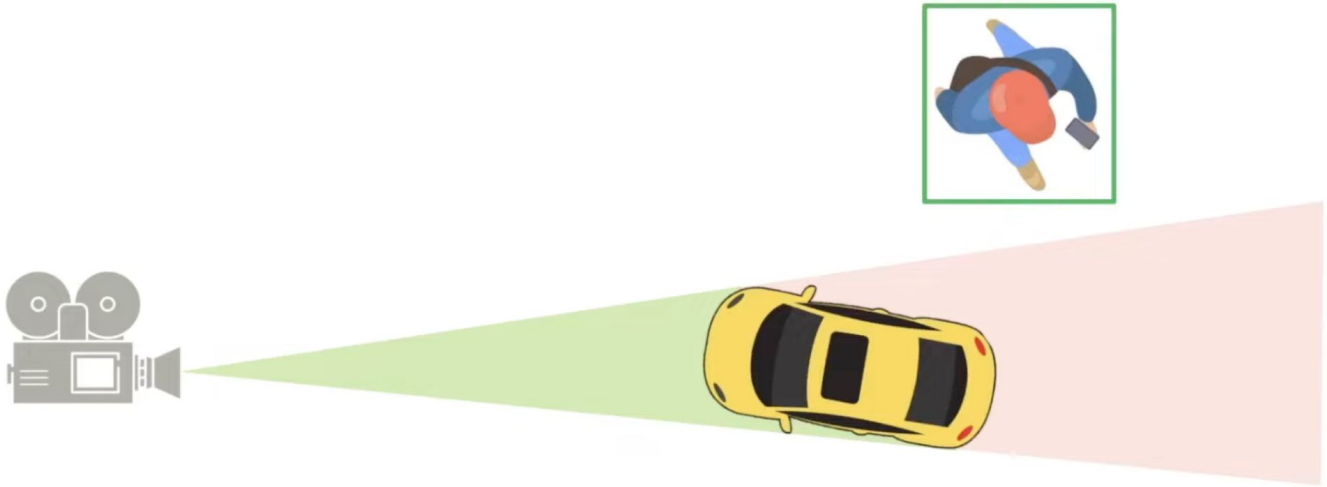
- Multi-object tracking suffers from long-term occlusions
  - Recall MOT



Quo Vadis: Is Trajectory Forecasting the Key Towards Long-Term Multi-Object Tracking? NeurIPS 2022

We can combine trajectory prediction with MOT…

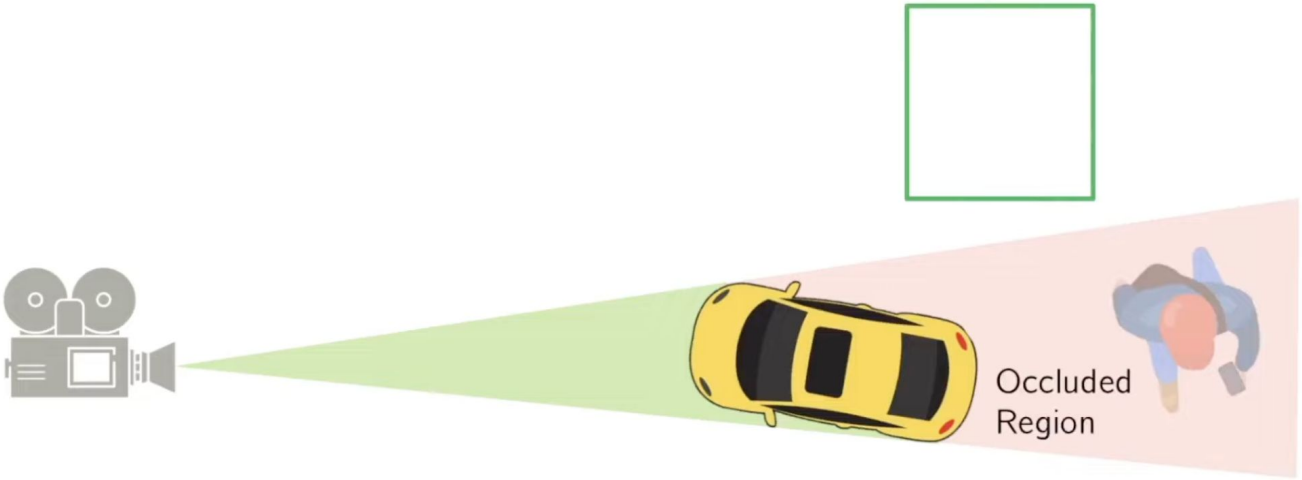
- Multi-object tracking suffers from long-term occlusions



Quo Vadis: Is Trajectory Forecasting the Key Towards Long-Term Multi-Object Tracking? NeurIPS 2022

We can combine trajectory prediction with MOT…

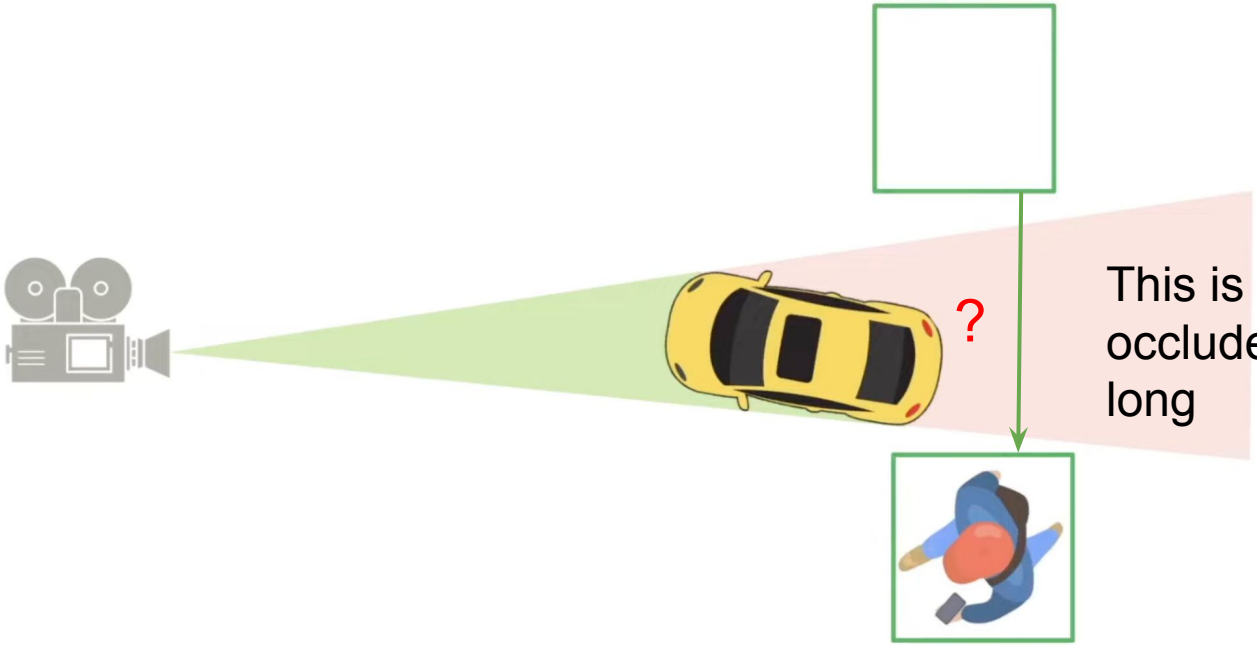
- Multi-object tracking suffers from long-term occlusions



Quo Vadis: Is Trajectory Forecasting the Key Towards Long-Term Multi-Object Tracking? NeurIPS 2022

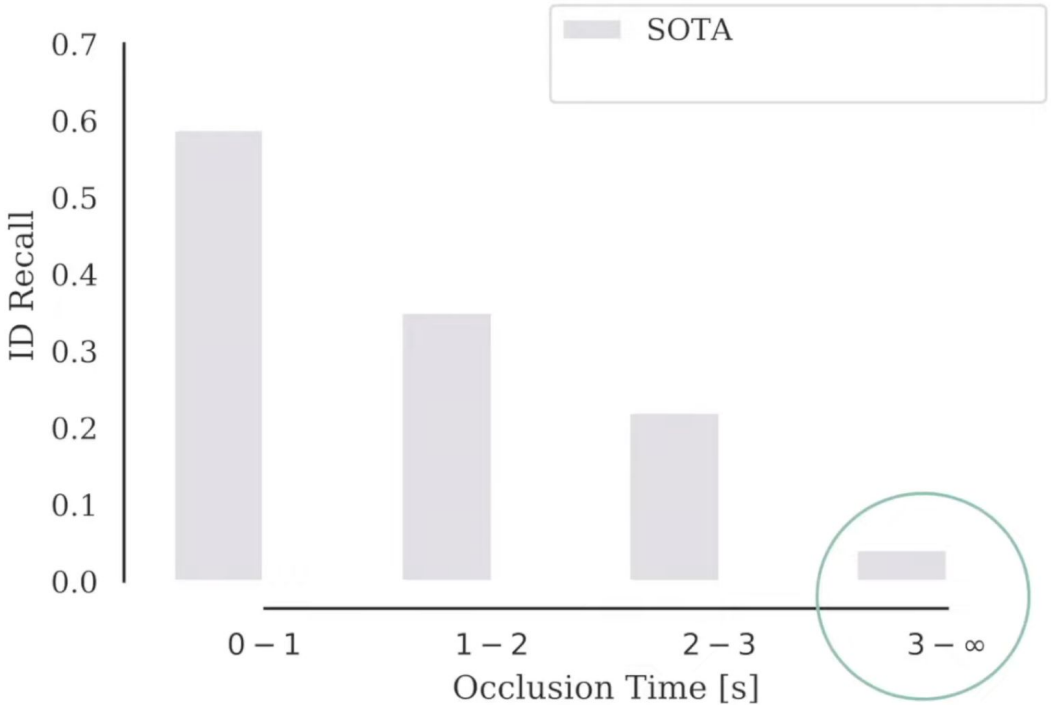
We can combine trajectory prediction with MOT…

- Multi-object tracking suffers from long-term occlusions



We can combine trajectory prediction with MOT…

- Multi-object tracking suffers from long-term occlusions



Performance of top-8-trackers on MOT17 benchmark

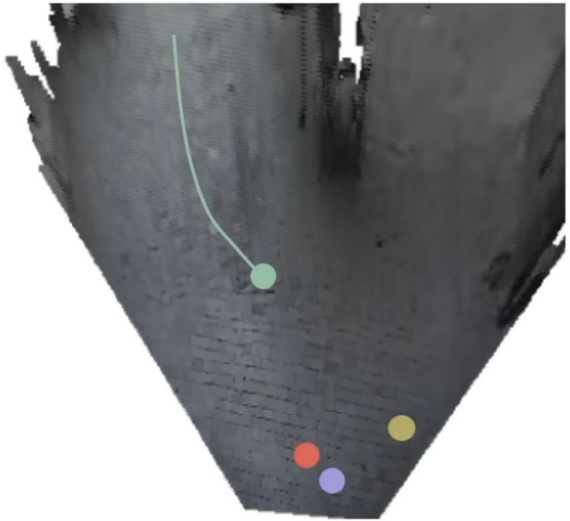
Long-term MOT with trajectory prediction

- Perform trajectory prediction in bird's view when occlusion in MOT occurs
  - Step 1. Project image and the detected person location into BEV view

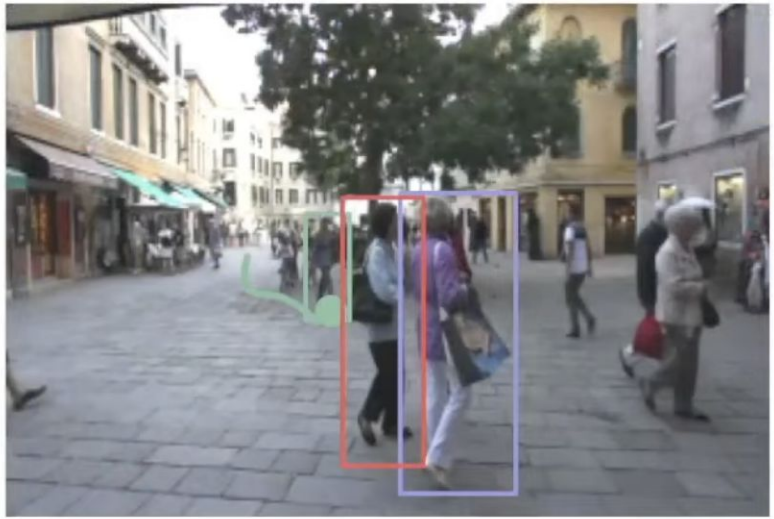
Here is the method:

This assumes everything is on the ground

Scene Bird's-Eye View



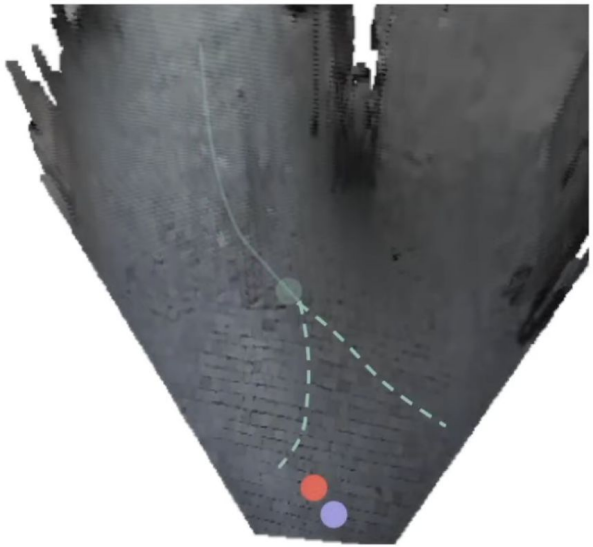
Scene Image



Long-term MOT with trajectory prediction

- Perform trajectory prediction in bird's view when occlusion in MOT occurs
  - Step 2. **Predict trajectory** when occlusion occurs
    - This essentially reduces the search space for re-identification of the person

Scene Bird's-Eye View



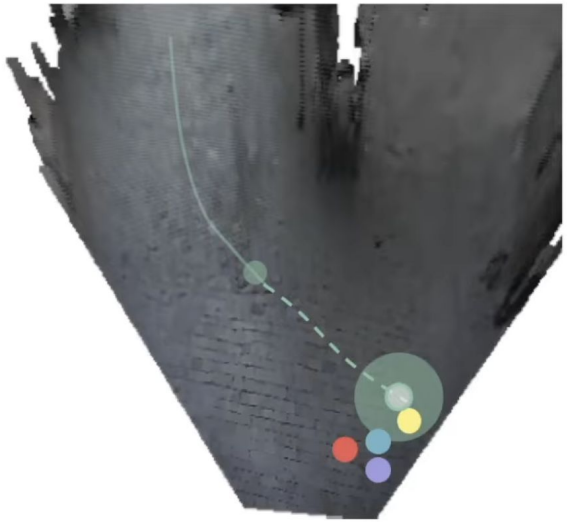
Scene Image



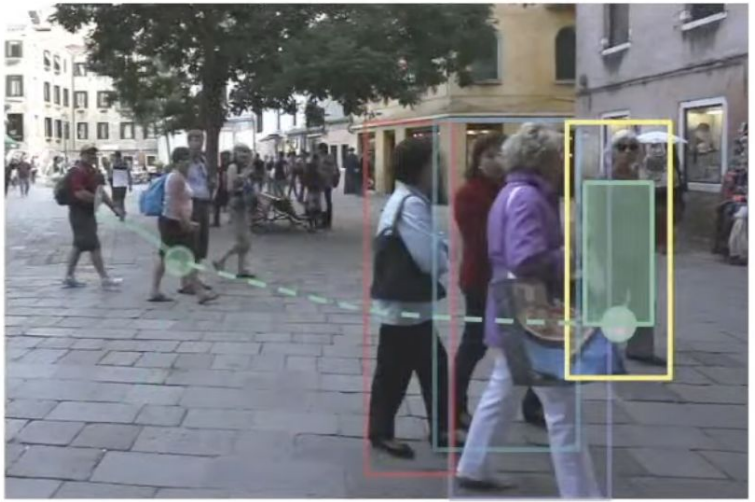
Long-term MOT with trajectory prediction

- Perform trajectory prediction in bird's view when occlusion in MOT occurs
  - Finally, use appearance matching when the person appears

Scene Bird's-Eye View

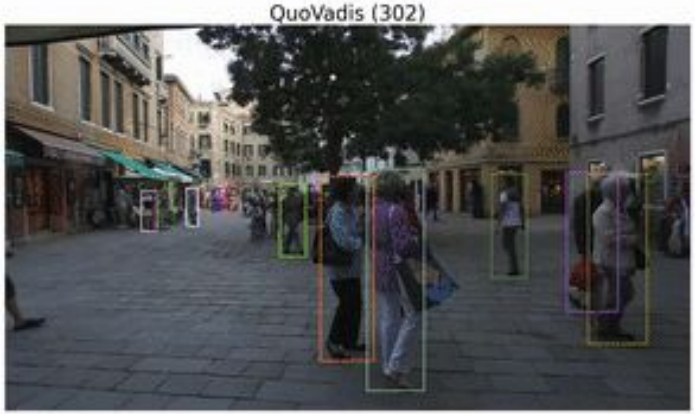


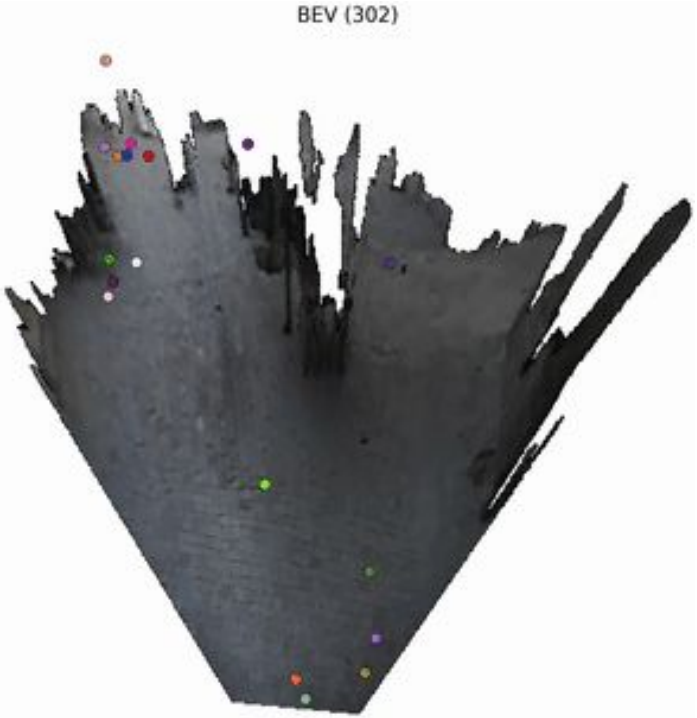
Scene Image



Long-term MOT with trajectory prediction

- Perform trajectory prediction in bird's view when occlusion in MOT occurs
  - Full example





Long-term MOT with trajectory prediction

- Experiments
  -

